

Claude Code Deep Dive Workshop

Chapter 5 - CLI Reference

전체 CLI 플래그, Headless 모드, CI/CD 자동화

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com



Agenda

Claude Code Workshop - Chapter 5 - CLI Reference

01

claude 명령 기본

플래그 카테고리, 기본 옵션, 도움말

02

Headless 모드 심화

대화형 없이 파이프라인 통합

03

출력 형식과 파싱

text/json/stream-json, jq 패턴

04

CI/CD 통합

GitHub Actions, GitLab, Jenkins

05

자동화 패턴

PR 리뷰, 로그 분석, 보고서 생성

06

환경변수와 디버깅

모든 환경변수, 트러블슈팅

07

Recap + 4 Labs

핵심 정리와 4가지 실습

CHAPTER 05

CLI Reference

전체 CLI 플래그, Headless 모드, CI/CD 자동화



Learning Objectives

CLI 마스터가 챕터를 마치고 할 수 있는 것

OBJ 1

CLI 전체 마스터

claude 명령의 모든 플래그를 정확히 이해하고 상황에 맞게 활용합니다.

OBJ 2

Headless 모드

대화형 없이 스크립트와 파이프라인에서 Claude를 활용합니다.

OBJ 3

출력 파싱

JSON, stream-json 출력을 jq로 파싱해 다른 도구와 연동합니다.

OBJ 4

CI/CD 통합

GitHub Actions, GitLab CI, Jenkins에 Claude를 통합해 자동화합니다.

OUTCOME

챕터를 마치면 Claude Code를 명령줄 자동화 도구로 활용해 PR 리뷰, 코드 분석, 보고서 생성 같은 작업을 무인 자동화할 수 있게 됩니다.

4

학습 목표

9 Parts

구성

4 Labs

실습 랩

CLI master

대상 수준

왜 CI인가

대화형에서 자동화로의 전환

REASON 1

재현 가능

동일한 명령은 동일한 결과를 만듭니다. 스크립트로 저장하고 공유 가능.

REASON 2

확장 가능

수천 파일 일괄 처리, 야간 배치, 병렬 실행이 명령 한 줄로 가능합니다.

REASON 3

통합 가능

pipe, 환경변수, exit code로 어떤 도구와도 결합. 표준 Unix 철학.

REASON 4

감사 가능

모든 호출이 로그로 남고 input/output을 추적 가능합니다.

PARADIGM SHIFT

Day 1-2는 사람이 Claude와 대화하는 패러다임이었다면, Chapter 5는 시스템이 Claude를 호출하는 패러다임입니다. 한 사람의 도구에서 팀과 인프라의 도구로 진화합니다.

Repeatable

재현

Scalable

확장

Composable

결합

Auditable

감사

CHAPTER 05 – PART 01

CLI Reference – Claude 명령 기본

전체 CLI 플래그, Headless 모드, CI/CD 자동화



Part 1 / claude 명령 기본

15장으로 전체 플래그 마스터

PART OBJECTIVE

claude 명령의 구조, 6대 플래그 카테고리, 자주 쓰는 옵션과 문제 회피 패턴을 학습합니다. 이 챕터 이후의 모든 내용의 기초가 됩니다.

TOPIC 1

명령 구조

대화형 vs Headless vs
pipe vs redirect 4가지 호
출 방식

TOPIC 2

플래그 6대 카테고리

동작, 출력, 권한, 모델, 컨텍
스트, 디버깅

TOPIC 3

실전 옵션

각 카테고리의 핵심 옵션을
풍부한 예시와 함께

TOPIC 4

실수 회피

따옴표, 변수 확장, stderr
같은 일반 실수 패턴 정리

15 Slides

Part 1 분량

~30 min

예상 강의 시간

6 categories

플래그 영역

Foundation

기초 마스터

claude 명령 구조

4가지 호출 방식

invocations

```
# 방식 1: 대화형 (interactive)
$ claude
> 이 코드를 리뷰해 줘
> ...

# 방식 2: Headless (single prompt)
$ claude -p "이 디렉토리의 README.md 요약해 줘"
# 한 번 응답 후 즉시 종료

# 방식 3: Pipe (stdin)
$ cat report.md | claude -p "이 보고서의 핵심 3가지 정리"
$ git diff | claude -p "이 변경사항을 한 줄로 요약"

# 방식 4: Redirect (stdout)
$ claude -p "Python 정렬 알고리즘 5개 비교" > comparison.md
$ claude -p "README 초안 작성" | tee README.md

# 조합 예시
$ tail -100 app.log | claude -p \
  "이 로그에서 에러를 찾고 원인 분석" \
  --output-format json > analysis.json

# 다중 명령
$ for dir in src/*/; do
  claude -p "$dir 폴더의 코드 품질 평가" > "reports/$(basename $dir).md"
done
```

Interactive

대화형

Headless

한 번 호출

Pipe

stdin 입력

Redirect

파일 저장

--help 시스템

문서를 명령으로 확인

1. 전체 도움말
\$ claude --help

Usage: claude [options] [prompt]

Options:

-p, --print <prompt>	Headless 모드 (한 번 응답)
--output-format <fmt>	text json stream-json
--model <model>	사용할 모델 ID
--max-turns <n>	최대 도구 호출 수
--max-tokens <n>	최대 응답 토큰
--allowed-tools <tools>	허용 도구 (coma 구분)
--disallowed-tools <tools>	차단 도구
--dangerously-skip-permissions	권한 확인 우회 (위험)
--cwd <dir>	작업 디렉토리
--system-prompt <text>	시스템 프롬프트 추가
--resume	마지막 세션 이어서
--continue	특정 세션 ID 이어서
--debug	디버그 로그
--version	버전 정보
--help	이 도움말

2. 하위 명령 도움말
\$ claude doctor --help
\$ claude mcp --help

3. 환경변수 정보
\$ claude doctor --verbose
모든 환경변수와 현재 값 표시

--help

--help

기본

subcmd --help

하위 명령

doctor

환경 진단

--version

버전

6대 플래그 카테고리

플래그를 머릿속에 정리하기

카테고리	대표 플래그	용도
동작 모드	-p, --print	Headless로 한 번 응답
출력 형식	--output-format	text/json/stream-json
권한	--allowed-tools, --disallowed-tools	도구 사용 통제
모델	--model, --max-turns, --max-tokens	모델과 제한 설정
컨텍스트	--cwd, --system-prompt, --resume	실행 환경 지정
디버깅	--debug, --verbose, doctor	문제 진단

6

카테고리

~20

주요 플래그

Layered

계층 구조

Composable

조합 가능

동작 모드 옵션

-p 와 인자 없음의 차이

-p 옵션

```
# 1. 대화형 (인자 없음)
$ claude
> 이 코드 리뷰해 줘
> 추가로 보안 관점에서도 봐 줘
> /exit
```

```
# 2. Headless 단일 응답
$ claude -p "현재 디렉토리 구조 분석"
# 응답 후 종료. 추가 대화 불가.
```

```
# 3. Headless 대화 도구 호출
$ claude -p "README.md를 읽고 요약해서 SUMMARY.md에 저장"
# Claude가 Read와 Write를 자동으로 호출
# 모든 작업 완료 후 종료
```

```
# 4. -p 와 --print 동일
$ claude -p "..."
$ claude --print "..."
```

```
# 5. 표준입력과 함께
$ git diff | claude -p "이 변경의 위험도 평가"
# stdin이 있으면 그것을 컨텍스트로 사용
```

```
# 6. 표준입력과 -p 동시
$ git log --oneline -20 | claude -p \
  "이 커밋 목록을 분류해 줘"
```

```
# 주의: 대화형에서는 stdin 사용 못 함
# Claude가 사용자 입력으로 stdin을 기대하기 때문
```

No -p
대화형

-p
Headless

pipe
stdin 입력

combined
조합

출력 형식 옵션

text vs json vs stream-json

```
# 1. text (기본값) - 사람이 읽기 좋음
$ claude -p "Python의 list comprehension 설명"
# 출력: 일반 마크다운 텍스트

# 2. json - 구조화 데이터 (스크립트용)
$ claude -p "..." --output-format json
{
  "type": "result",
  "subtype": "success",
  "result": "...",
  "session_id": "ses_abc123",
  "usage": {
    "input_tokens": 152,
    "output_tokens": 384
  },
  "duration_ms": 2840
}
```

```
# 3. stream-json - 실시간 스트림 (UI/CI용)
$ claude -p "..." --output-format stream-json
{"type":"system","subtype":"init","session_id":"..."}
{"type":"message","role":"assistant","content":"Pyt"}
{"type":"message","role":"assistant","content":"hon "}
{"type":"tool_use","tool":"Read","input":{"...}}
{"type":"tool_result","content":"..."}
{"type":"result","subtype":"success","duration_ms":2840}
```

--output-format

사용 패턴

- # - text: 사람 읽기, 빠른 확인
- # - json: 스크립트 파싱, 결과만 필요할 때
- # - stream-json: 실시간 진행 표시, 토큰별 처리

text

기본

json

스크립트

stream-json

UI/CI

jq pair

jq 파싱

JSON 출력 파싱

jq 6가지 핵심 패턴

jq patterns

```
# 1. 결과만 추출
$ claude -p "... " --output-format json | jq -r '.result'

# 2. 토큰 사용량 추출
$ claude -p "... " --output-format json | \
  jq '.usage | {in: .input_tokens, out: .output_tokens}'

# 3. 응답 시간 측정 (밀리초 → 초)
$ claude -p "... " --output-format json | \
  jq '.duration_ms / 1000'

# 4. 결과를 다음 명령에 전달
$ RESULT=$(claude -p "Python 정렬 알고리즘 추천" \
  --output-format json | jq -r '.result')
$ echo "$RESULT" > recommendation.md
```

```
# 5. 비용 계산 (단가 가정)
$ claude -p "... " --output-format json | jq '
  .usage as $u |
  ($u.input_tokens * 0.003 + $u.output_tokens * 0.015) / 1000
'

# 6. 에러 확인
$ RESPONSE=$(claude -p "... " --output-format json)
$ SUBTYPE=$(echo "$RESPONSE" | jq -r '.subtype')
$ if [ "$SUBTYPE" != "success" ]; then
  ERROR=$(echo "$RESPONSE" | jq -r '.error // "unknown"')
  echo "Failed: $ERROR" >&2
  exit 1
fi
```

.result

결과 추출

.usage

토큰

.duration_ms

시간

.subtype

에러 체크

권한 옵션

도구 사용 통제

permissions

```
# 1. --allowed-tools : 화이트리스트 방식
$ claude -p "이 디렉토리 분석" \
  --allowed-tools Read,Grep,Glob

# 2. --disallowed-tools : 블랙리스트 방식
$ claude -p "..." \
  --disallowed-tools Write>Edit,Bash

# 3. 명령 패턴 (Bash 세분화)
$ claude -p "..." \
  --allowed-tools "Read,Grep,Bash(git diff:*)"

# 4. settings.json과 결합
# settings.json의 deny는 항상 우선
# allowed-tools는 settings의 allow에 추가됨
```

```
# 5. 위험한 옵션: --dangerously-skip-permissions
$ claude -p "..." --dangerously-skip-permissions
# 모든 권한 확인 우회 (CI/CD에서만 사용)
# 실수로 prod 환경에 사용하면 큰 사고

# 6. 조합 패턴 (CI 환경)
$ claude -p "PR 리뷰만 작성" \
  --allowed-tools "Read,Grep,Glob,Bash(gh pr:*),Bash(git diff:*)" \
  --disallowed-tools "Write>Edit,Bash(git push:*)" \
  --max-turns 20

# Permissions 우선순위
# deny > 명시 disallowed > 명시 allowed > settings allow
```

--allowed
화이트리스트

--disallowed
블랙리스트

--skip
위험 옵션

Layered
계층 적용

모델 옵션

--model, --max-turns, --max-tokens

```
# 1. --model : 모델 선택
$ claude -p "..." --model claude-sonnet-4-5 # 기본
$ claude -p "..." --model claude-haiku-4-5 # 빠르고 저렴
$ claude -p "..." --model claude-opus-4-5 # 최고 성능

# settings.json의 modelAlias 사용 가능
$ claude -p "..." --model fast # → Haiku
$ claude -p "..." --model smart # → Opus

# 2. --max-turns : 최대 도구 호출 횟수
$ claude -p "프로젝트 전체 분석" --max-turns 10
# 도구를 10회 호출하면 종료 (무한 루프 방지)
# 기본값: 30, 권장: 작업 복잡도에 맞춰 조정

# 3. --max-tokens : 응답 토큰 제한
$ claude -p "이 코드 한 줄 평" --max-tokens 200
# 응답이 200 토큰을 넘으면 잘림
# 짧은 응답을 강제할 때 유용
```

```
# 4. 비용 절감 패턴
# - 간단한 작업: Haiku (10x 저렴)
# - 표준 작업: Sonnet (기본, 균형)
# - 복잡한 분석: Opus (10x 비싸지만 정확)

# 5. CI 환경 권장 설정
$ claude -p "..." \
  --model claude-haiku-4-5 \
  --max-turns 20 \
  --max-tokens 4000 \
  --output-format json
```

model

--model

모델 선택

--max-turns

도구 제한

--max-tokens

응답 제한

Cost ctrl

비용 통제

컨텍스트 옵션

--cwd, --system-prompt, --resume

context

```
# 1. --cwd : 작업 디렉토리 변경
$ claude -p "이 프로젝트 분석" --cwd /path/to/project
# 임시로 다른 디렉토리에서 실행
# 현재 셸의 cwd는 변경되지 않음
```

```
# 2. --system-prompt : 시스템 프롬프트 추가
$ claude -p "이 코드 검토해 줘" \
  --system-prompt "당신은 보안 전문가입니다. \
    OWASP Top 10 관점에서 검토."
```

```
# 또는 파일에서 로드
$ claude -p "..." \
  --system-prompt "$(cat prompts/security-reviewer.md)"
```

```
# 3. --resume : 마지막 세션 이어가기
$ claude -p "그 다음 단계는?" # 새 세션
$ claude --resume # 마지막 대화로 복귀
$ claude --resume -p "그 다음" # 마지막 + Headless
```

```
# 4. --continue : 특정 세션 이어가기
$ claude --continue ses_abc123 # session_id로 복귀
# Headless에서 다단계 작업에 유용
```

```
# 5. 세션 ID 확인
$ claude -p "..." --output-format json | jq -r '.session_id'
```

```
# 6. 다단계 자동화 패턴
$ STEP1=$(claude -p "코드 분석" --output-format json)
$ SES=$(echo "$STEP1" | jq -r '.session_id')
$ STEP2=$(claude --continue "$SES" -p "이제 수정안 제시")
```

--cwd

작업 디렉토리

--system-prompt

시스템 추가

--resume

마지막 이어

--continue

ID로 이어

디버깅 옵션

--debug, --verbose, doctor

debug

```
# 1. --debug : 상세 디버그 로그
$ claude -p "..." --debug 2> debug.log
# stderr에 모든 내부 동작 로그
# Tool 호출, MCP 통신, 모델 응답까지 추적
```

```
# 2. --verbose : 사용자 향 상세 출력
$ claude doctor --verbose
# 환경 정보, 설정 파일 위치, MCP 연결 상태
# 환경변수 현재 값까지 모두 표시
```

```
# 3. claude doctor : 종합 진단
$ claude doctor
```

```
Claude Code v1.2.0
✓ Node.js v20.10.0
✓ Settings: ~/.claude/settings.json (valid)
✓ Authentication: API key (sk-ant-...)
✓ MCP servers: 3 connected
✓ Network: api.anthropic.com reachable
✗ Disk space: 92% used (warning)
```

```
# 4. 환경변수 확인
$ env | grep -i claude
$ env | grep -i anthropic
```

```
# 5. 버전 정보
$ claude --version
claude-code 1.2.0 (build 2026.05.20)
```

```
# 6. 일반 디버깅 흐름
1. claude doctor      # 기본 진단
2. claude --version   # 버전 확인
3. --debug 재실행      # 상세 로그
4. /mcp logs <server> # MCP 문제
5. Slack #claude 문의  # 에스컬레이션
```

--debug

내부 로그

doctor

종합 진단

--verbose

상세 출력

--version

버전

자주 하는 실수

4가지 함정 회피

MISTAKE 1

따옴표 누락

claude -p 이 코드 리뷰 →
"이 코드 리뷰".
공백 포함 프롬프트는 따옴표
필수.

MISTAKE 2

변수 미확장

claude -p \$VAR → '...' (작은따옴표 안에서는 변수 확장 안 됨).
큰따옴표 사용.

MISTAKE 3

stderr 누락

--debug 시 stderr는 별도.
2>&1 또는 2>file.log로 캡처.
그냥 잡으면 못 봄.

MISTAKE 4

모델명 오타

claude-sonnet-4.5 (잘못) vs
claude-sonnet-4-5 (정확).
하이픈 사용.

CHECKLIST

문제 발생 시 우선 echo로 명령을 확인하고, --debug로 내부 동작을 보고, claude doctor로 환경을 점검.

90% 이상은 셸 인용이나 변수 확장 문제.

Quoting

인용

Expansion

확장

stderr

에러 통로

Typo

오타

환경변수 개요

주요 환경변수 8가지

환경변수	용도	기본값
ANTHROPIC_API_KEY	API 인증 키	settings에서 로드
CLAUDE_CODE_USE_BEDROCK=1	AWS Bedrock 경로 사용	미설정
CLAUDE_CODE_USE_VERTEX=1	GCP Vertex AI 경로 사용	미설정
AWS_REGION	Bedrock 리전	us-east-1
HTTPS_PROXY	프록시 서버 (사내망)	미설정
NODE_EXTRA_CA_CERTS	사내 CA 인증서	미설정
CLAUDE_CODE_LOG_LEVEL	로그 레벨 (debug/info/warn)	info
CLAUDE_CODE_DISABLE_TELEMETRY	텔레메트리 비활성화	미설정

8 vars

주요 변수

Provider

경로 선택

Proxy

사내망

Audit

감사

명령 조합 패턴

실전에서 자주 쓰는 조합

patterns

```
# 패턴 1: 빠른 일회성 분석
$ claude -p "이 파일의 핵심 아이디어 3가지"
```

```
# 패턴 2: pipe로 다른 명령 결과 분석
$ git log --since="1 week ago" --oneline | \
  claude -p "이번 주 작업 카테고리별 분류"
```

```
# 패턴 3: JSON 출력 후 jq 파싱
$ claude -p "README 요약" --output-format json | \
  jq -r '.result' > summary.md
```

```
# 패턴 4: 안전한 CI 설정
$ claude -p "PR 변경 사항 요약" \
  --allowed-tools "Read,Bash(gh pr:*)" \
  --max-turns 10 \
  --output-format json | \
  jq -r '.result'
```

```
# 패턴 5: 환경변수로 동적 프롬프트
$ PR_NUM=$(gh pr list --json number -q '.[0].number')
$ claude -p "PR #$PR_NUM 리뷰" --output-format json
```

```
# 패턴 6: 다단계 세션
$ S=$(claude -p "분석" --output-format json | \
  jq -r '.session_id')
$ claude --continue "$S" -p "수정안 제시"
$ claude --continue "$S" -p "테스트 작성"
```

```
# 패턴 7: 병렬 처리
$ for f in src/*.js; do
  claude -p "$f 분석" > "reports/$(basename $f).md" &
done
$ wait # 모든 작업 완료 대기
```

7 patterns

실전

Pipe

결합

Parallel

병렬

Reusable

재사용

Part 1 Recap

claude 명령 기본 핵심 정리

TAKEAWAY 1

4가지 호출 방식

대화형, Headless (-p),
pipe (stdin), redirect
(stdout) 활용

TAKEAWAY 2

6대 플래그

동작, 출력, 권한, 모델, 컨텍
스트, 디버깅 카테고리 마스
터

TAKEAWAY 3

JSON 파싱

jq로 .result, .usage,
.duration_ms, .subtype
추출

TAKEAWAY 4

실수 회피

큰따옴표 사용, stderr 캡처
, 모델명 정확히, doctor 진
단

NEXT - PART 2

Part 2는 Headless 모드 심화입니다. 자동화 스크립트 작성, 큰 입력 처리, 세션 재사용, 에러 처리를 15장에 걸쳐 다룹니다.

Part 1

완료

Part 2

Headless (15p)

Part 3

출력 파싱 (15p)

~3h

잔여

CHAPTER 05 – PART 02

CLI Reference – Headless 모드

전체 CLI 플래그, Headless 모드, CI/CD 자동화



Part 2 / Headless 모드 심화

대화형 없이 자동화에 통합

PART OBJECTIVE

Headless 모드의 모든 입력 방식, 큰 입력 처리, 다단계 세션, 에러 처리, 자동화 스크립트 5종을 학습합니다. 실무 자동화의 핵심입니다.

TOPIC 1

입력 방식 6종

인자, pipe, heredoc, 리다이렉트, 다중 명령 결합

TOPIC 2

큰 입력 처리

대용량 파일 분할,
Subagent 위임,
pagination 전략

TOPIC 3

세션과 에러

다단계 세션 재사용,
retry+backoff, 에러 분류

TOPIC 4

5가지 자동화

PR 리뷰, 로그 분석, 문서 생성,
번역, 검증

15 Slides

Part 2 분량

~30 min

예상 강의 시간

5 scripts

완성 코드

Automation

자동화 핵심

Headless 입력 방식 6종

모든 입력 패턴 정리

input methods

```
# 1. 인자 (가장 단순)
$ claude -p "이 디렉토리 분석"
```

```
# 2. stdin pipe
$ cat report.md | claude -p "이 보고서 핵심 추출"
```

```
# 3. heredoc (큰 텍스트)
$ claude -p "$(cat <<'EOF'
다음 코드를 리뷰해 주세요:
- 가독성
- 성능
- 보안
EOF
)"
```

```
# 4. 리다이렉트로 출력
$ claude -p "주간 보고서 작성" > weekly.md
$ claude -p "..." 2> debug.log      # stderr만
$ claude -p "..." &> all.log         # 둘 다
```

```
# 5. 결합: 다른 명령의 결과를 컨텍스트로
$ (echo "## 최근 commit"; git log -10 --oneline) | \
  claude -p "이 commit 패턴 분석"
```

```
# 6. 다중 명령 (순차)
$ claude -p "코드 분석" > analysis.md && \
  claude -p "$(cat analysis.md) 기반 개선안" > improvements.md
```

```
# 7. xargs로 일괄 처리 (병렬 X)
$ find . -name "*.py" | \
  xargs -I{} claude -p "{} 분석" > all-analysis.txt
```

```
# 8. GNU parallel로 병렬 처리
$ find . -name "*.py" | \
  parallel -j 4 'claude -p "{} 분석" > "reports/{/}.md"'
```

arg
단순

pipe
stdin

heredoc
큰 입력

parallel
병렬

Exit Code 활용

결과 분기 패턴

```
# Claude의 exit code
# 0 - 성공
# 1 - 일반 오류
# 2 - 인자 오류
# 124 - timeout
# 130 - SIGINT (Ctrl+C)
# 137 - SIGKILL (메모리 초과 등)

# 패턴 1: 성공/실패 분기
$ claude -p "테스트 실행" && echo "OK" || echo "Failed"

# 패턴 2: exit code 캡처
$ claude -p "..."; EC=$?
$ if [ $EC -eq 0 ]; then
    echo "성공"
elif [ $EC -eq 124 ]; then
    echo "Timeout"
else
    echo "에러 코드: $EC"
fi
```

```
# 패턴 3: timeout 설정 (5분 제한)
$ timeout 300 claude -p "큰 작업"
# 5분 초과 시 exit code 124

# 패턴 4: 응답 안에서 결과 판단
$ RESULT=$(claude -p "이 코드 안전한가? yes/no만 응답" \
  --output-format json | jq -r '.result')
$ if [ "$RESULT" = "yes" ]; then
    echo "안전"
else
    echo "위험"
    exit 1
fi

# 패턴 5: 재시도 로직
$ MAX=3
$ for i in $(seq 1 $MAX); do
    claude -p "..." && break
    [ $i -lt $MAX ] && sleep $((2**i))
done
```

exit code

0
성공

124
Timeout

\$?
캡처

Retry
재시도

자동화 1 / PR 리뷰

pr-review.sh 완성 스크립트

```
#!/bin/bash
# pr-review.sh - GitHub PR 자동 리뷰
set -euo pipefail

PR_NUM="${1:?Usage: pr-review.sh <PR-number>}"

# 1. PR 정보 수집
PR_TITLE=$(gh pr view "$PR_NUM" --json title -q '.title')
PR_DIFF=$(gh pr diff "$PR_NUM")

# 2. Claude로 리뷰
REVIEW=$(claude -p "$(cat <<EOF
다음 PR을 리뷰해 주세요.

제목: $PR_TITLE

변경 내용:
$PR_DIFF
```

검토 관점:

1. 코드 품질 (가독성, 명명, 복잡도)
2. 보안 (OWASP Top 10)
3. 테스트 커버리지
4. 성능 영향

출력: Markdown, 우선순위별 (Critical/Important/Suggestion)

```
EOF
)" \
  --allowed-tools "Read,Grep,Glob" \
  --max-turns 10 \
  --output-format json | jq -r '.result')
```

3. PR에 코멘트 게시

```
echo "$REVIEW" | gh pr comment "$PR_NUM" --body-file -
```

```
echo "✓ PR #$PR_NUM 리뷰 완료"
```

사용 예시

```
# $ ./pr-review.sh 1234
```

pr-review.sh

gh CLI
GitHub

heredoc
프롬프트

JSON+jq
파싱

auto comment
게시

자동화 2 / 대량 로그 분석

analyze-logs.sh 병렬 처리

```
#!/bin/bash
# analyze-logs.sh - 로그 파일 병렬 분석
set -euo pipefail
LOG_DIR="${1:-./logs}"
OUTPUT_DIR="${2:-./analyses}"
mkdir -p "$OUTPUT_DIR"
# 각 로그 파일 분석 함수
analyze_one() {
    local file="$1"
    local out="$OUTPUT_DIR/${basename "$file"}.log.md"
    claude -p "$(cat <<EOF
```

다음 로그에서:

1. 에러 패턴 (빈도순 top 5)
2. 시간대별 분포
3. 의심스러운 동작
4. 권장 조치

```
로그 내용:
$(tail -10000 "$file")
EOF
)" \
    --model claude-haiku-4-5 \
    --max-turns 5 \
    --output-format json | jq -r '.result' > "$out"
```

```
echo "✓ $(basename "$file")"
}
export -f analyze_one
export OUTPUT_DIR
# GNU parallel로 4개씩 병렬 실행
find "$LOG_DIR" -name "*.log" | \
parallel -j 4 analyze_one {}
# 통합 보고서
{
    echo "# 로그 분석 통합 보고서"
    echo "분석 일시: $(date)"
    for f in "$OUTPUT_DIR"/*.md; do
        echo ""
        echo "## $(basename "$f" .md)"
        cat "$f"
    done
} > "$OUTPUT_DIR/SUMMARY.md"
```

```
echo "✓ 총 $(ls "$OUTPUT_DIR" | wc -l)개 분석 완료"
```

analyze-logs.sh

parallel

병렬

Haiku

저비용

tail

큰 파일

Summary

통합

자동화 3 / 문서 생성

generate-docs.sh API 문서

```
#!/bin/bash
# generate-docs.sh - API 문서 자동 생성
set -euo pipefail

SRC_DIR="${1:?Usage: generate-docs.sh <src-dir>}"
OUTPUT="docs/API.md"

mkdir -p docs

# 1. 모든 API 라우터 파일 수집
ROUTES=$(find "$SRC_DIR" -name "routes.js" -o -name "*Controller.ts" | head -50)

if [ -z "$ROUTES" ]; then
    echo "라우터 파일을 찾지 못함" >&2
    exit 1
fi

# 2. Claude에게 문서 생성 요청
claude -p "$(cat <<EOF
다음 API 라우터 파일들을 분석해서 종합 API 문서를 작성하세요.

대상 파일:
$ROUTES
```

generate-docs.sh

문서 구성:

1. 개요 (API 카테고리 분류)
2. 인증 방법
3. 엔드포인트 (HTTP 메서드, 경로, 인자, 응답, 예시)
4. 에러 코드
5. Rate Limiting
6. 변경 이력

각 엔드포인트는 다음 형식으로:

```
\\`\\`\\`
```

```
### POST /api/users
```

사용자 생성

```
**Request:**
```

```
{ "name": "...", "email": "..." }
```

```
**Response (201):**
```

```
{ "id": "...", "name": "...", "createdAt": "..." }
```

```
\\`\\`\\`
```

```
EOF
```

```
)" \
```

```
--allowed-tools "Read,Grep,Glob" \
```

```
--max-turns 30 \
```

```
--output-format json | jq -r '.result' > "$OUTPUT"
```

```
echo "✓ API 문서 생성: $OUTPUT ($(wc -l < $OUTPUT) lines)"
```

find

파일 수집

heredoc

명세

max-turns 30

다단계

Markdown

출력

자동화 4 / 문서 번역

translate-docs.sh 점진 번역

```
#!/bin/bash
# translate-docs.sh - 마크다운 문서 한/영 번역
set -euo pipefail

FILE="${1:?Usage: translate-docs.sh <file> <ko|en>}"
LANG="${2:?Usage: translate-docs.sh <file> <ko|en>}"
GLOSSARY="${GLOSSARY:-~/\.claude/glossary.md}"

# 출력 파일명 결정
DIR=$(dirname "$FILE")
BASE=$(basename "$FILE" .md)
OUT="$DIR/${BASE}.${LANG}.md"

# 사내 용어 사전 로드
GLOSSARY_CONTENT=""
if [ -f "$GLOSSARY" ]; then
    GLOSSARY_CONTENT=$(cat "$GLOSSARY")
fi
```

```
# 번역 실행 (점진 - 큰 파일도 처리)
claude -p "$(cat <<EOF
다음 마크다운 문서를 ${LANG}로 번역하세요.
원칙:
- 자연스러운 톤 (직역 금지)
- 코드 블록과 명령어는 유지
- 마크다운 구조 보존
- 사내 용어 사전 적용
사내 용어 사전:
$GLOSSARY_CONTENT
원본:
$(cat "$FILE")
번역된 마크다운만 출력하세요 (다른 설명 X).
EOF
)" \
--max-turns 5 \
--output-format json | jq -r '.result' > "$OUT"

# 검증: 라인 수가 너무 다른면 경고
ORIG_LINES=$(wc -l < "$FILE")
TRANS_LINES=$(wc -l < "$OUT")
if [ $((ORIG_LINES - TRANS_LINES)) -gt 10 ] || \
  [ $((TRANS_LINES - ORIG_LINES)) -gt 10 ]; then
    echo "⚠ 라인 수 차이 큼: 원본 $ORIG_LINES vs 번역 $TRANS_LINES" >&2
fi
echo "✓ 번역 완료: $OUT"
```

translate-docs.sh

glossary

용어

Markdown

구조 보존

Validation

검증

Bilingual

양방향

자동화 5 / 코드 검증

validate-code.sh pre-commit

```
#!/bin/bash
# validate-code.sh - 변경 코드 검증 (pre-commit hook용)
set -euo pipefail

# Git에서 staged 파일 가져옴
STAGED=$(git diff --cached --name-only --diff-filter=ACM | \
    grep -E '\.(js|ts|py|go)$' || true)

if [ -z "$STAGED" ]; then
    echo "검증할 코드 파일 없음"
    exit 0
fi

echo "검증 대상: $STAGED"

# 각 파일 검증
FAILED=0
for file in $STAGED; do
    RESULT=$(claude -p "$(cat <<EOF
다음 파일을 검증하세요. 문제가 있으면 PROBLEM, 없으면 OK 한 단어로 응답.
```

```
검증 항목:
- 시크릿/토큰 노출
- 명백한 보안 취약점
- 데이터 손실 위험

파일 ($file):
$(cat "$file")
EOF
)" \
    --allowed-tools "Read" \
    --max-turns 3 \
    --output-format json | jq -r '.result' | head -1)
if [ "$RESULT" != "OK" ]; then
    echo "x $file: $RESULT"
    FAILED=$((FAILED + 1))
else
    echo "✓ $file"
fi
done
if [ $FAILED -gt 0 ]; then
    echo "❌ $FAILED개 파일 검증 실패. commit 차단" >&2
    exit 1
fi
echo "✓ 모든 파일 검증 통과"

# .git/hooks/pre-commit에 심플릭 링크
# $ ln -s ../../validate-code.sh .git/hooks/pre-commit
```

validate-code.sh

pre-commit
혹

staged only
대상 좁힘

Boolean
OK/PROBLEM

Block
커밋 차단

큰 입력 처리

컨텍스트 한계 극복 전략

large input

```
# 문제: 컨텍스트 윈도우는 약 200K 토큰
# 대용량 파일/디렉토리는 한 번에 안 들어감

# 전략 1: split으로 분할
$ split -l 1000 large.log chunk_
$ for f in chunk_*; do
    claude -p "이 청크 요약" < "$f" >> summary.md
done
$ # 마지막에 모든 요약을 종합
$ claude -p "$(cat summary.md) 통합 결론" > final.md

# 전략 2: Subagent에 위임
$ claude -p "다음 디렉토리 전체를 분석. 큰 파일은 \
Subagent에게 위임하고 결과만 받아 종합 보고서 작성. \
대상: ./src"

# 전략 3: tail/head로 부분만
$ tail -5000 huge.log | claude -p "최근 패턴 분석"
$ head -100 huge.csv | claude -p "이 CSV 스키마 추론"
```

```
# 전략 4: 사전 처리 (압축/요약)
$ grep ERROR huge.log | claude -p "에러 패턴 분류"
$ # 1GB 로그 → 10MB 에러만 → claude 입력 가능

# 전략 5: 점진적 처리 (세션 재사용)
$ S=$(claude -p "디렉토리 구조 파악" --output-format json \
| jq -r '.session_id')
$ claude --continue "$S" -p "이제 src/ 하위 자세히"
$ claude --continue "$S" -p "이제 tests/ 검토"

# 전략 6: 코드 검색으로 좁히기
$ grep -rn "function handlePayment" src/ | \
    claude -p "이 함수의 호출 패턴 분석"
```

split

분할

Subagent

위임

tail/head

부분

session

점진

세션 재사용

--resume과 --continue

```
# --resume : 마지막 세션 자동 복귀
$ claude --resume
> 그 다음 단계는? # 이전 컨텍스트 그대로

# --resume과 -p 조합
$ claude --resume -p "그 다음은?"

# --continue <session_id> : 특정 세션 명시
$ claude --continue ses_abc123 -p "추가 분석"

# 세션 ID 추출
$ S=$(claude -p "초기 분석" --output-format json \
  | jq -r '.session_id')
$ echo "Session: $S"

# 다단계 자동화 예시
# Step 1: 코드 분석
S=$(claude -p "src/ 디렉토리 종합 분석" \
  --allowed-tools "Read,Grep,Glob" \
  --output-format json | jq -r '.session_id')
```

```
# Step 2: 같은 컨텍스트에서 개선안
claude --continue "$S" -p "이제 구체적인 개선안 5개" \
  --max-turns 10 > improvements.md

# Step 3: 우선순위 선정
claude --continue "$S" -p "위 5개 중 ROI 높은 3개 선정" \
  > priorities.md

# Step 4: 구현 가이드
claude --continue "$S" -p "선정된 3개의 구체적 구현 가이드" \
  --allowed-tools "Read,Write" \
  --max-turns 20

# 세션 만료
# 약 30분간 활동 없으면 자동 만료
# 만료 후 --continue는 새 세션 시작
```

session reuse

--resume

마지막

--continue

ID 명시

jq session_id

추출

Multi-step

다단계

에러 처리와 재시도

run_claude 함수

```
# 견고한 호출을 위한 함수
run_claude() {
    local prompt="$1"
    local max_retries=3
    local retry=0
    local delay=2

    while [ $retry -lt $max_retries ]; do
        # JSON 출력으로 호출
        local response
        response=$(claude -p "$prompt" \
            --output-format json \
            --max-turns 20 \
            --max-tokens 4000 \
            2> "/tmp/claude-err-$$log")
        local rc=$?
        # 성공: subtype 0 | success
        if [ $rc -eq 0 ]; then
            local subtype
            subtype=$(echo "$response" | jq -r '.subtype // "unknown"')
            if [ "$subtype" = "success" ]; then
                echo "$response"
                return 0
            fi
        fi
    fi
}
```

```
# 실패: 재시도
retry=$((retry + 1))
if [ $retry -lt $max_retries ]; then
    echo "Retry $retry/$max_retries (delay ${delay}s)" >&2
    sleep $delay
    delay=$((delay * 2)) # exponential backoff
fi
done
# 모든 재시도 실패
echo "Failed after $max_retries attempts" >&2
cat "/tmp/claude-err-$$log" >&2
rm -f "/tmp/claude-err-$$log"
return 1
}
# 사용
if RESPONSE=$(run_claude "이 코드 분석"); then
    echo "$RESPONSE" | jq -r '.result' > analysis.md
else
    echo "분석 실패" >&2; exit 1
fi
```

run_claude

Function

재사용

Retry x 3

재시도

Backoff

지수 대기

stderr log

로그

로깅과 관측성

log_claude 함수

```
# 모든 호출을 구조화 JSON 로그로 남김
log_claude() {
    local prompt="$1"
    local task_name="${2:-unnamed}"
    local start_ts=$(date +%s%3N)
    local response
    response=$(claude -p "$prompt" --output-format json)
    local rc=$?
    local end_ts=$(date +%s%3N)
    local elapsed_ms=$((end_ts - start_ts))
```

```
# JSON 로그 (한 줄)
local log_entry
log_entry=$(jq -nc \
    --arg ts "$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
    --arg task "$task_name" \
    --arg user "$(whoami)" \
    --arg host "$(hostname)" \
    --argjson elapsed "$elapsed_ms" \
    --argjson rc "$rc" \
    --argjson resp "$response" \
    '{
        ts: $ts, task: $task, user: $user, host: $host,
        elapsed_ms: $elapsed,
        exit_code: $rc,
        session_id: $resp.session_id,
        duration_ms: $resp.duration_ms,
        usage: $resp.usage,
        subtype: $resp.subtype
    }')
```

```
# 사내 로그 시스템 전송 (비동기)
echo "$log_entry" >> "/var/log/claude/$(date +%Y-%m-%d).log"
{
    curl -s -X POST "$LOG_ENDPOINT" \
        -H "Content-Type: application/json" \
        -d "$log_entry"
} &
# 호출자에게 응답 전달
echo "$response"
return $rc

# 사용 예시
RESPONSE=$(log_claude "PR 리뷰" "pr-review")
```

log_claude

Structured
JSON

Async POST
비동기

Metadata
컨텍스트

Audit
감사

완성 스크립트 예시

4단계 일일 보고서

```
#!/bin/bash
# daily-report.sh - 매일 자동 실행되는 일일 보고서
set -euo pipefail
DATE=$(date -d "yesterday" +%Y-%m-%d)
WORKDIR="/tmp/daily-$DATE"
mkdir -p "$WORKDIR"

# Stage 1: 데이터 수집
echo "[1/4] 데이터 수집..."
gh pr list --state merged --search "merged:$DATE" \
  --json title,author,labels > "$WORKDIR/prs.json"
gh issue list --state closed --search "closed:$DATE" \
  --json title,author > "$WORKDIR/issues.json"
git log --since="$DATE 00:00" --until="$DATE 23:59" \
  --pretty=format:'%h %an %s' > "$WORKDIR/commits.log"

# Stage 2: AI 분석 (세션 재사용)
echo "[2/4] AI 분석..."
S=$(claude -p "다음 데이터에서 어제의 작업 패턴 분석: \
  $(cat $WORKDIR/prs.json $WORKDIR/issues.json)" \
  --model claude-haiku-4-5 \
  --output-format json | jq -r '.session_id')
```

```
# Stage 3: 요약 생성
echo "[3/4] 요약 생성..."
claude --continue "$S" \
  -p "위 분석을 기반으로 일일 보고서 (Markdown) 작성. \
  구성: 어제 완료, 진행 중, 차단 사항, 오늘 계획" \
  --output-format json | jq -r '.result' > "$WORKDIR/report.md"

# Stage 4: Slack 게시
echo "[4/4] Slack 게시..."
curl -X POST "$SLACK_WEBHOOK" \
  -H 'Content-Type: application/json' \
  -d "$(jq -n --rawfile r "$WORKDIR/report.md" '{text:$r}')"

echo "✓ 일일 보고서 완료"

# Cron 등록 (매일 오전 9시)
# 0 9 * * * /opt/daily-report.sh >> /var/log/daily-report.log 2>&1
```

daily-report.sh

gh+git

데이터

--continue

재사용

Slack

게시

cron

스케줄

성능 팁

Headless 자동화 최적화

TIP 1

병렬 실행

GNU parallel로 -j 4 정도.
너무 많으면 rate limit. 4-8이 권장.

TIP 2

결과 캐싱

같은 입력의 결과를 hash로 캐싱.
두 번째 호출은 즉시 응답.

TIP 3

모델 적합

간단한 분류는 Haiku, 복잡한 분석은 Sonnet, 핵심만 Opus.

TIP 4

입력 최소화

필요한 부분만 추출 (grep, awk, jq)해서 컨텍스트 토큰 절약.

MEASUREMENT

모든 호출의 duration_ms와 usage를 로깅하면 어디서 시간/비용이 많이 들어가는지 한눈에 보입니다. 그 부분부터 최적화하세요.

Parallel

병렬

Cache

재사용

Right model

적합 모델

Minimal

최소 입력

Part 2 Recap

Headless 모드 핵심 정리

TAKEAWAY 1

입력 방식

인자, pipe, heredoc, 리다이렉트, parallel 활용

TAKEAWAY 2

5 자동화

PR 리뷰, 로그 분석, 문서 생성, 번역, 검증 완성

TAKEAWAY 3

큰 입력

split, Subagent, tail/head, 사전 처리, 세션 재사용

TAKEAWAY 4

운영

재시도 + backoff, 구조화 로깅, 성능 측정

NEXT - PART 3

Part 3은 출력 형식과 파싱 심화입니다. JSON 스키마, stream-json 이벤트, jq 고급 패턴, 결과 후처리를 15장에 걸쳐 다룹니다.

Part 2

완료

Part 3

출력 파싱 (15p)

Part 4

CI/CD (20p)

~2.5h

잔여

CHAPTER 05 - PART 03

CLI Reference – 출력형식과 파싱심화

전체 CLI 플래그, Headless 모드, CI/CD 자동화



Part 3 / 출력 형식과 파싱 심화

JSON, stream-json 마스터

PART OBJECTIVE

JSON 응답 스키마 전체, stream-json 이벤트 타입, 고급 jq 패턴, 결과 후처리(CSV/HTML 변환), 다중 호출 집계를 학습합니다.

TOPIC 1

스키마 이해

JSON 응답의 모든 필드와
stream-json 이벤트 타입
전체

TOPIC 2

jq 고급 패턴

select, group_by, map,
reduce 같은 고급 jq 활용

TOPIC 3

결과 후처리

Markdown → HTML 변환
, JSON → CSV 변환, 대시
보드 데이터

TOPIC 4

집계와 분석

다중 호출 결과를 통합하고
토큰 사용량/비용 계산

15 Slides

Part 3 분량

~30 min

예상 강의 시간

jq master

학습 목표

Pipeline

파이프라인

JSON 응답 스키마 전체

모든 필드와 의미

JSON schema

```
# 성공 응답 (subtype: success)
{
  "type": "result",
  "subtype": "success",
  "result": "...실제 응답 텍스트...",
  "session_id": "ses_01HXYZ...",
  "duration_ms": 2840,
  "model": "claude-sonnet-4-5",
  "usage": {
    "input_tokens": 1523,
    "output_tokens": 487,
    "cache_creation_input_tokens": 0,
    "cache_read_input_tokens": 0
  },
  "stop_reason": "end_turn",
  "num_turns": 3,
  "is_error": false
}
```

```
# 에러 응답 (subtype: error)
{
  "type": "result",
  "subtype": "error",
  "error": "Tool 'WebFetch' not allowed",
  "error_code": "permission_denied",
  "session_id": "ses_01HXYZ...",
  "duration_ms": 120,
  "is_error": true
}

# 일부 응답 (subtype: max_turns_reached)
{
  "type": "result",
  "subtype": "max_turns_reached",
  "result": "...부분 응답...",
  "session_id": "...",
  "num_turns": 30,
  "is_error": false
}
```

```
# subtype 종류
# - success          정상 완료
# - error            도구/권한/내부 오류
# - max_turns_reached --max-turns 도달
# - max_tokens_reached --max-tokens 도달
```

result

핵심 텍스트

usage

토큰

duration_ms

시간

subtype

결과 분류

stream-json 이벤트 타입

실시간 처리를 위한 이벤트

stream events

```
# stream-json은 한 줄 당 하나의 JSON 이벤트
```

```
# 이벤트 1: system (세션 시작)
```

```
{"type": "system", "subtype": "init", "session_id": "ses...",  
  "model": "claude-sonnet-4-5", "cwd": "/path/to/project"}
```

```
# 이벤트 2: message (모델 응답 청크)
```

```
{"type": "message", "role": "assistant",  
  "content": [{"type": "text", "text": "이 코드를 분석..."}],  
  "stop_reason": null}
```

```
# 이벤트 3: tool_use (도구 호출 시작)
```

```
{"type": "tool_use", "id": "toolu_abc",  
  "name": "Read",  
  "input": {"file_path": "/path/to/file.js"}}
```

```
# 이벤트 4: tool_result (도구 호출 결과)
```

```
{"type": "tool_result", "tool_use_id": "toolu_abc",  
  "content": "...파일 내용...",  
  "is_error": false}
```

```
# 이벤트 5: text_delta (스트리밍 텍스트 청크)
```

```
{"type": "text_delta", "delta": "분"}  
{"type": "text_delta", "delta": "석"}  
{"type": "text_delta", "delta": " 결과"}
```

```
# 이벤트 6: result (최종 완료)
```

```
{"type": "result", "subtype": "success",  
  "result": "전체 응답...", "duration_ms": 2840,  
  "usage": {...}}
```

```
# 처리 패턴
```

```
$ claude -p "..." --output-format stream-json | \n  while IFS= read -r line; do\n    TYPE=$(echo "$line" | jq -r '.type')\n    case "$TYPE" in\n      tool_use) echo "\n\ ${echo "$line" | jq -r '.name'}" ;;\n      text_delta) echo -n "${echo "$line" | jq -r '.delta'}" ;;\n      result) echo "\n 완료" ;;\n    esac\n  done
```

6 types

이벤트

system

시작

tool_*

도구

result

종료

jq 기초 복습

핵심 연산자 10가지

연산자	용도	예시
.	입력 전체	echo '{"a":1}' jq .
.field	필드 추출	jq .usage
.[]	배열 펼치기	jq .items[]
	파이프 (다음 단계)	jq .a length
-r	raw 출력 (따옴표 제거)	jq -r .result
select(...)	조건 필터	select(.is_error)
map(...)	배열 변환	map(.name)
group_by(.)	그룹화	group_by(.category)
reduce	누적 계산	reduce .[] as \$x (0; .+\$x)
@csv, @tsv	형식 변환	map([.a,.b]) @csv

10 ops

핵심

Composable

조합

Streaming

스트림

JSON-native

JSON 전용

jq 고급 / 필터링

select와 조건 표현식

```
# 1. 단순 조건 필터
$ claude -p "..." --output-format json | \
  jq 'select(.is_error == false)'
```

```
# 2. 복합 조건 (AND/OR)
$ jq 'select(.usage.input_tokens > 1000 and .duration_ms < 5000)'
$ jq 'select(.subtype == "error" or .is_error == true)'
```

```
# 3. 정규식 매칭
$ jq 'select(.result | test("ERROR|FATAL"))'
$ jq 'select(.model | startswith("claude-opus"))'
```

```
# 4. null 체크 (안전한 접근)
$ jq 'select(.usage?.input_tokens // 0 > 500)'
$ # ? : 필드 없으면 null
$ # // : null이면 기본값
```

```
# 5. 배열 안에서 필터
$ echo ' [{ "a":1},{ "a":2},{ "a":3}] ' | \
  jq 'map(select(.a > 1))'
# 결과: [{ "a":2},{ "a":3}]
```

```
# 6. 다중 조건 분기
$ jq 'if .is_error then "FAIL"
    elif .subtype == "max_turns_reached" then "PARTIAL"
    else "OK"
    end'
```

```
# 7. 응답 분류
$ for response in responses/*.json; do
  STATUS=$(jq -r '
    if .is_error then "fail"
    elif .duration_ms > 10000 then "slow"
    else "ok"
    end' "$response")
  echo "response: $STATUS"
done
```

jq filter

select

조건

and/or

복합

test()

정규식

if/then

분기

jq 고급 / 집계

group_by, reduce, 통계

aggregation

시나리오: 100개 응답에서 통계 추출

1. 모든 응답을 배열로 합치기 (slurp)

```
$ jq -s '.' responses/*.json > all.json
```

-s : 모든 입력을 배열로 합침

2. 평균 응답 시간

```
$ jq '[](duration_ms) | add / length' all.json
```

3. 토큰 사용량 합계

```
$ jq '[](usage.input_tokens) | add' all.json
```

```
$ jq '[](usage.output_tokens) | add' all.json
```

4. 모델별 그룹화 + 카운트

```
$ jq 'group_by(.model) | map({  
  model: .[0].model,  
  count: length,  
  avg_duration: ([](duration_ms) | add / length),  
  total_in: ([](usage.input_tokens) | add),  
  total_out: ([](usage.output_tokens) | add)  
})' all.json
```

5. 에러율 계산

```
$ jq '{  
  total: length,  
  errors: []( | select(.is_error)) | length  
} | .error_rate = (.errors / .total * 100)' all.json
```

6. p95 응답 시간 (백분위)

```
$ jq '[](duration_ms) | sort | .[((length * 0.95) | floor)]' \
```

all.json

7. reduce로 누적

```
$ jq 'reduce .[] as $x (  
  {count:0, sum:0};  
  {count: (.count+1), sum: (.sum+$x.duration_ms)}  
) | .avg = (.sum / .count)' all.json
```

-s slurp

배열화

group_by

그룹

p95

백분위

reduce

누적

JSON → CSV 변환

스프레드시트로 분석

JSON to CSV

시나리오: 100개 응답을 CSV로 변환해 Excel 분석

1. 기본 CSV 변환

```
$ jq -r '[
  .timestamp,
  .model,
  .duration_ms,
  .usage.input_tokens,
  .usage.output_tokens,
  .is_error
] | @csv' all.json > responses.csv
```

2. 헤더 추가

```
$ {
  echo '"timestamp","model","duration_ms","input","output","error"'
  jq -r '[
    (.timestamp // ""),
    (.model // ""),
    (.duration_ms // 0),
    (.usage.input_tokens // 0),
    (.usage.output_tokens // 0),
    .is_error
  ] | @csv' all.json
} > responses.csv
```

3. 다중 응답 → CSV (slurp 사용)

```
$ jq -rs '
["timestamp","model","duration_ms","input","output","error"],
[.] | [
  .timestamp,
  .model,
  .duration_ms,
  .usage.input_tokens,
  .usage.output_tokens,
  .is_error
]
)
| @csv
' responses/*.json > responses.csv
```

4. TSV (탭 구분)

```
$ jq -rs '...| @tsv' all.json > responses.tsv
```

5. 비용 컬럼 추가

```
$ jq -r '. as $r |
($r.usage.input_tokens * 0.003 +
 $r.usage.output_tokens * 0.015) / 1000 as $cost |
[$r.timestamp, $r.model, $r.duration_ms, $cost] | @csv
' all.json > with-cost.csv
```

6. Excel에서 바로 열기

```
$ open responses.csv # macOS
$ start responses.csv # Windows
```

@csv

변환

Header

헤더

-s slurp

여러 파일

Cost calc

비용

Markdown → HTML 변환

Claude 응답을 웹으로

MD to HTML

시나리오: Claude의 Markdown 응답을 HTML 리포트로

1. pandoc로 변환 (가장 쉬움)

```
$ claude -p "주간 보고서 작성" --output-format json | \
jq -r '.result' | \
pandoc -f markdown -t html5 -s \
--metadata title="Weekly Report" \
-o report.html
```

2. CSS 스타일 적용

```
$ claude -p "..." --output-format json | jq -r '.result' | \
pandoc -f markdown -t html5 -s \
--css style.css \
--highlight-style tango \
-o report.html
```

3. 템플릿 사용

```
$ claude -p "..." --output-format json | jq -r '.result' > body.md
$ cat <<EOF > report.html
<!DOCTYPE html>
<html><head>
<title>Report - $(date +%Y-%m-%d)</title>
<style>body { max-width: 800px; margin: auto; }</style>
</head><body>
$(pandoc body.md -t html5)
</body></html>
EOF
```

4. node로 변환 (marked 라이브러리)

```
$ claude -p "..." --output-format json | jq -r '.result' | \
node -e "
const marked = require('marked');
const stdin = require('fs').readFileSync(0, 'utf8');
console.log(marked.parse(stdin));
" > report.html
```

5. 메타데이터 포함

```
$ RESP=$(claude -p "..." --output-format json)
$ TOKENS=$(echo "$RESP" | jq '.usage.output_tokens')
$ DURATION=$(echo "$RESP" | jq '.duration_ms')
$ {
echo "<!-- Tokens: $TOKENS, Duration: ${DURATION}ms -->"
echo "$RESP" | jq -r '.result' | pandoc -f markdown -t html5
} > report.html
```

pandoc

도구

CSS

스타일

Template

템플릿

Metadata

추적

다중 호출 집계

batch 처리와 결과 통합

```
#!/bin/bash
# batch-analyze.sh - 여러 파일 분석 후 통합
mkdir -p results
# 1. 각 파일을 개별 분석 (병렬)
analyze_one() {
    local f="$1"
    local out="results/${basename "$f"}.json"
    claude -p "다음 코드를 분석. 결과는 JSON: \
{complexity: 1-10, issues: [], suggestions: []}" \
--output-format json \
< "$f" > "$out"
}
export -f analyze_one
find src/ -name "*.js" | parallel -j 4 analyze_one
```

```
# 2. 결과 집계 (jq slurp)
jq -s '
# 모든 분석 결과의 .result를 JSON으로 파싱
map(.result | fromjson) as $analyses |

{
    total_files: ($analyses | length),
    avg_complexity: ([($analyses[].complexity) | add / length),
    high_complexity: [
        $analyses[] | select(.complexity > 7) |
        {file: input_filename, complexity}
    ],
    total_issues: ([($analyses[].issues[]) | length),
    common_issues: ([($analyses[].issues[])
        | group_by(.)
        | map({issue: .[0], count: length})
        | sort_by(-count) | .[0:5])
    ]
}
' results/*.json > summary.json
```

```
# 3. Markdown 리포트로 변환
jq -r '
"# 코드 품질 분석 결과
" +
"
분석 파일: \(.total_files)개" +
"
평균 복잡도: \(.avg_complexity)" +
"
총 이슈: \(.total_issues)개" +
"
## 자주 발견된 이슈 Top 5
" +
"([.common_issues[] | "- \(.issue) (\(.count)건)" ] | join("
"))
' summary.json > REPORT.md

echo "✓ 보고서: REPORT.md"
```

aggregation

parallel

병렬 분석

fromjson

JSON 파싱

group_by

집계

Markdown

리포트

토큰 사용량과 비용 추적

운영 메트릭

```
# Anthropic API 단가 (예시, 1M 토큰당 USD)
# claude-haiku-4-5 : in $1.00, out $5.00
# claude-sonnet-4-5 : in $3.00, out $15.00
# claude-opus-4-5 : in $15.00, out $75.00
```

1. 단일 호출 비용 계산

```
$ jq -s '
  .usage as $u |
  if .model | startswith("claude-haiku") then
    ($u.input_tokens * 1.0 + $u.output_tokens * 5.0) / 1e6
  elif .model | startswith("claude-sonnet") then
    ($u.input_tokens * 3.0 + $u.output_tokens * 15.0) / 1e6
  elif .model | startswith("claude-opus") then
    ($u.input_tokens * 15.0 + $u.output_tokens * 75.0) / 1e6
  else 0 end
' response.json
# 결과: 0.0023 (약 $0.0023)
```

2. 일일 비용 합계 (모든 응답 파일)

```
$ jq -s '
  map(.usage as $u |
    if .model | startswith("claude-haiku") then
      ($u.input_tokens * 1.0 + $u.output_tokens * 5.0) / 1e6
    elif .model | startswith("claude-sonnet") then
      ($u.input_tokens * 3.0 + $u.output_tokens * 15.0) / 1e6
    elif .model | startswith("claude-opus") then
      ($u.input_tokens * 15.0 + $u.output_tokens * 75.0) / 1e6
    else 0 end)
  | add' /var/log/claude/$(date +%Y-%m-%d)/*.json
```

3. 모델별 토큰 사용량 (보고용)

```
$ jq -s '
  group_by(.model) | map({
    model: .[0].model,
    calls: length,
    total_in: ([.[].usage.input_tokens] | add),
    total_out: ([.[].usage.output_tokens] | add)
  })' logs/*.json
```

4. CloudWatch/Datadog 메트릭 전송

```
$ COST=$(jq '...' response.json)
$ aws cloudwatch put-metric-data \
  --namespace ClaudeCode \
  --metric-name DailyCost \
  --value "$COST"
```

cost tracking

Per-call

호출별

Daily

일일 합산

Per-model

모델별

Metric

대시보드

출력 파일 분할

여러 파일로 결과 저장

시나리오: 한 응답에서 여러 파일을 추출

1. 도구 호출별 결과 저장

```
$ claude -p "src/의 모든 .js 파일을 분석. \
각 파일별 분석 결과를 JSON으로 출력" \
--output-format stream-json | \
while IFS= read -r line; do
    TYPE=$(echo "$line" | jq -r '.type')
    if [ "$TYPE" = "tool_result" ]; then
        FILE=$(echo "$line" | jq -r '.tool_use_id')
        echo "$line" | jq '.content' > "results/$FILE.json"
    fi
done
```

2. 응답 안의 코드 블록 추출

```
$ claude -p "Python 경렬 알고리즘 5개. \
각각을 별도 함수로" --output-format json | \
jq -r '.result' | \
awk '/```python/{p=1; n++; next}
/```/{p=0; next}
p{print > "alg_" n ".py"}'
```

3. 섹션별 파일 분할 (마크다운)

```
$ claude -p "API 문서 작성. ## 섹션 단위로 구성" \
--output-format json | jq -r '.result' | \
awk '/^## /{f=tolower($2); gsub(/ /,"_",f); f="docs/"f".md"}
f{print > f}'
```

4. JSON 응답을 여러 파일로 (jq)

```
$ echo '[{"id":1,"data":"..."}, {"id":2,"data":"..."}]' | \
jq -c '.[ ]' | \
while IFS= read -r obj; do
    ID=$(echo "$obj" | jq -r '.id')
    echo "$obj" > "items/$ID.json"
done
```

5. zip으로 묶기

```
$ zip -r analysis-results.zip results/ docs/
```

split output

tool_result

도구별

awk

섹션 분할

jq -c

JSON 분할

zip

묶기

실시간 진행 표시

stream-json으로 UI

progress

```
#!/bin/bash
# progress.sh - 실시간 진행 표시
```

```
claude -p "큰 작업: 모든 .py 파일 분석 후 리포트 생성" \
--output-format stream-json | \
while IFS= read -r line; do
  TYPE=$(echo "$line" | jq -r '.type')
  case "$TYPE" in
    system)
      MODEL=$(echo "$line" | jq -r '.model')
      echo "🔥 Started (model: $MODEL)"
      ;;
    tool_use)
      NAME=$(echo "$line" | jq -r '.name')
      INPUT=$(echo "$line" | jq -c '.input')
      echo "🔗 $NAME: $INPUT"
      ;;
    tool_result)
      IS_ERR=$(echo "$line" | jq -r '.is_error')
      if [ "$IS_ERR" = "true" ]; then
        echo "❌ 도구 실패"
      else
        echo "✓ 결과 수신"
      fi
      ;;
  esac
done
```

```
text_delta)
  DELTA=$(echo "$line" | jq -r '.delta')
  echo -n "$DELTA"
  ;;
result)
  SUBTYPE=$(echo "$line" | jq -r '.subtype')
  DURATION=$(echo "$line" | jq '.duration_ms')
  TOKENS=$(echo "$line" | jq '.usage.output_tokens')
  echo ""
  echo "=====
  echo "✓ 완료 [$SUBTYPE]"
  echo "  시간: ${DURATION}ms"
  echo "  토큰: $TOKENS"
  ;;
esac
done
```

```
# 결과 예시:
# 🔥 Started (model: claude-sonnet-4-5)
# 🔗 Read: {"file_path": "src/app.py"}
# ✓ 결과 수신
# 🔗 Read: {"file_path": "src/utils.py"}
# ✓ 결과 수신
# 분석 결과...
# =====
# ✓ 완료 [success]
```

system

시작

tool_use

도구 표시

text_delta

응답 흐름

result

완료

HTML 대시보드 생성

운영 메트릭 시각화

```
#!/bin/bash
# generate-dashboard.sh - 일일 사용량 대시보드

DATA=$(jq -s '
  map({
    ts: .timestamp,
    model: .model,
    duration: .duration_ms,
    in: .usage.input_tokens,
    out: .usage.output_tokens,
    cost: (
      if (.model | startswith("claude-sonnet")) then
        (.usage.input_tokens * 3 + .usage.output_tokens * 15) / 1e6
      else 0 end
    )
  })' /var/log/claude/$(date +%Y-%m-%d)/*.json)

cat <<HTML > dashboard.html
<!DOCTYPE html>
<html><head>
<title>Claude Code Dashboard - $(date +%Y-%m-%d)</title>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<style>
```

```
body { font-family: sans-serif; max-width: 1200px; margin: auto; }
.cards { display: grid; grid-template-columns: repeat(4, 1fr); gap: 1rem; }
.card { padding: 1rem; border: 1px solid #ddd; border-radius: 8px; }
canvas { max-height: 300px; }
</style>
</head><body>
<h1>📊 일일 사용량 - $(date +%Y-%m-%d)</h1>
<div class="cards">
  <div class="card">
    <h3>총 호출</h3>
    <p id="total"></p>
  </div>
  <div class="card">
    <h3>총 비용</h3>
    <p id="cost"></p>
  </div>
</div>
<canvas id="chart"></canvas>
<script>
const data = $DATA;
document.getElementById('total').textContent = data.length;
document.getElementById('cost').textContent =
  '$' + data.reduce((s,r) => s + r.cost, 0).toFixed(2);
// Chart.js 그래프 추가 가능
</script>
</body></html>
HTML
echo "✓ dashboard.html 생성"
```

dashboard

Chart.js

시각화

HTML inject

데이터

Cards

UI

Auto-gen

자동

출력 파싱 일반 실수

4가지 함정

MISTAKE 1

jq -r 누락

따옴표 포함된 문자열이 변수에 들어가 다음 명령이 깨짐. -r로 raw 사용.

MISTAKE 2

null 미처리

필드가 없을 수 있는데 직접 접근.
항상 // 0 또는 ? 사용.

MISTAKE 3

JSON 안 JSON

.result 안에 JSON 문자열일 때 fromjson 필요.
잊으면 문자열 처리.

MISTAKE 4

stream-json 한 줄씩

한 번에 전체 파싱하려 하면 실패.
while read로 한 줄씩 처리.

CHECKLIST

복잡한 jq 표현식은 jq -r 옵션과 null safety로 시작하세요. 디버깅 시 작은 단위로 쪼개서 테스트하고 echo와 함께 단계별로 검증합니다.

-r

raw 출력

//, ?

null 안전

fromjson

중첩 JSON

while

스트림

Part 3 Recap

출력 형식과 파싱 핵심 정리

TAKEAWAY 1

스키마 완전 이해

JSON 응답 모든 필드,
stream-json 6가지 이벤트
타입

TAKEAWAY 2

jq 고급 패턴

select, group_by,
reduce, fromjson으로 통
계와 변환

TAKEAWAY 3

변환과 후처리

JSON → CSV, Markdown
→ HTML, 다중 결과 통합

TAKEAWAY 4

운영 활용

비용 추적, 실시간 진행,
HTML 대시보드 자동 생성

NEXT - PART 4

Part 4는 CI/CD 통합입니다. GitHub Actions, GitLab CI, Jenkins에 Claude를 통합하는 완성된 워크플로를 20장에 걸쳐 다룹니다.

Part 3

완료

Part 4

CI/CD (20p)

Part 5

Patterns (20p)

~2h

잔여

CHAPTER 05 – PART 04

CLI Reference – CICD 통합

전체 CLI 플래그, Headless 모드, CI/CD 자동화



Part 4 / CI/CD 통합

파이프라인에 AI 자동화

PART OBJECTIVE

GitHub Actions, GitLab CI, Jenkins에 Claude Code를 통합합니다. 인증 패턴, 시크릿 관리, 워크플로 예시, 베스트 프랙티스를 모두 학습합니다.

TOPIC 1

GitHub Actions

workflow YAML,
secrets, matrix,
reusable workflow

TOPIC 2

GitLab CI

.gitlab-ci.yml, variables,
rules, runner 설정

TOPIC 3

Jenkins

Jenkinsfile, credentials,
declarative pipeline

TOPIC 4

베스트 프랙티스

시크릿 관리, 비용 통제, 보
안, 롤백

20 Slides

Part 4 분량

~40 min

예상 강의 시간

3 platforms

CI 시스템

Production

실무 적용

CI 통합 기본 패턴

3가지 핵심 요소

STEP 1

설치

`npm install -g @anthropic-ai/claude-code.`
CI runner에 매번 설치 또는 캐싱.

STEP 2

인증

ANTHROPIC_API_KEY를 secret으로.
Bedrock 경로도 동일한 패턴.

STEP 3

호출

Headless 모드(-p)에 `--output-format json`. JSON 파싱 후 후속 액션.

```
# 모든 CI 시스템에 공통된 기본 명령 형태
$ claude -p "PR #${PR_NUMBER} 리뷰" \
  --allowed-tools "Read,Grep,Glob,Bash(gh pr:*)" \
  --max-turns 15 \
  --output-format json | jq -r '.result'
```

common pattern

Install

설치

Auth

인증

Call

호출

Standard

표준화

GitHub Actions / 인증 설정

Secrets와 환경 분리

```
# 1. GitHub Secrets 설정
# Repository → Settings → Secrets and variables → Actions
# Name: ANTHROPIC_API_KEY
# Value: sk-ant-api03-...
```

```
# 2. Organization Secret (여러 repo 공유)
# Organization → Settings → Secrets → Actions
# - 모든 또는 특정 repo에 자동 적용
```

```
# 3. Environment Secret (환경별 분리)
# Repository → Settings → Environments
# - production: API_KEY_PROD
# - staging: API_KEY_STAGE
```

```
# 4. Bedrock 사용 시
# AWS_ROLE_ARN # OIDC로 임시 자격증명
# AWS_REGION
# CLAUDE_CODE_USE_BEDROCK=1
```

```
# 5. workflow에서 사용
# .github/workflows/example.yml
jobs:
  review:
    runs-on: ubuntu-latest
    environment: production # environment secret 사용
    steps:
      - uses: actions/checkout@v4
      - name: Setup
        run: |
          npm install -g @anthropic-ai/claude-code
      - name: Call Claude
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          claude -p "리뷰" --output-format json
```

GH auth

Repo
저장소

Org
조직

Env
환경

OIDC
AWS/GCP

GitHub Actions / 기본 워크플로

PR 리뷰 자동화

```
# .github/workflows/pr-review.yml
name: PR Review with Claude

on:
  pull_request:
    types: [opened, synchronize, reopened]

permissions:
  pull-requests: write
  contents: read

jobs:
  review:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0      # 전체 git history 필요
```

```
- name: Setup Node + Claude
  uses: actions/setup-node@v4
  with:
    node-version: '20'
  - run: npm install -g @anthropic-ai/claude-code

- name: Run Claude review
  id: review
  env:
    ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
    GH_TOKEN: ${ github.token }
    PR_NUMBER: ${ github.event.pull_request.number }
  run: |
    REVIEW=$(claude -p "$(cat .github/prompts/review.md)
    PR: #$PR_NUMBER
    $(gh pr diff $PR_NUMBER)" \
    --allowed-tools "Read,Grep,Glob,Bash(gh pr:*)" \
    --max-turns 15 \
    --output-format json | jq -r '.result')
    echo "$REVIEW" > review.md
    # GitHub Actions output에 저장
    {
      echo 'review<<EOF'
      echo "$REVIEW"
      echo EOF
    } >> $GITHUB_OUTPUT
```

```
- name: Post comment
  uses: marocchino/sticky-pull-request-comment@v2
  with:
    message: ${ steps.review.outputs.review }
```

pr-review.yml

on PR
트리거

permissions
권한

sticky comment
게시

GH_TOKEN
gh CLI

GitHub Actions / Matrix 빌드

여러 차원 병렬 실행

```
# .github/workflows/multi-analysis.yml
name: Multi-dimension Analysis

on: [push]

jobs:
  analyze:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        dimension: [security, performance, accessibility, seo]
        node: [18, 20]
        include:
          - dimension: security
            prompt-file: prompts/security.md
            allowed-tools: "Read,Grep,Glob"
          - dimension: performance
            prompt-file: prompts/performance.md
            allowed-tools: "Read,Grep,Bash(npm run benchmark)"

    name: ${ matrix.dimension }}-node${ matrix.node }}
```

```
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with:
      node-version: ${ matrix.node }}
  - run: npm install -g @anthropic-ai/claude-code
  - name: Run analysis
    env:
      ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }}
    run: |
      claude -p "$(cat ${ matrix.prompt-file })" \
        --allowed-tools "${ matrix.allowed-tools }}" \
        --output-format json > result.json
  - name: Upload artifact
    uses: actions/upload-artifact@v4
    with:
      name: ${ matrix.dimension }}-node${ matrix.node }}
      path: result.json

aggregate:
  needs: analyze
  runs-on: ubuntu-latest
  steps:
    - uses: actions/download-artifact@v4
    - run: ls -la
      # 모든 차원의 결과 통합 처리
```

matrix

matrix

병렬

fail-fast

독립 실행

include

맞춤

artifact

결과

GitHub Actions / Reusable Workflow

여러 repo에서 공유

```
# 1. 호출되는 워크플로 (재사용 가능)
# .github/workflows/claude-review.yml in org/.github
name: Claude Review (Reusable)
on:
  workflow_call:
    inputs:
      prompt-file:
        required: true
        type: string
      allowed-tools:
        required: false
        type: string
        default: "Read,Grep,Glob"
      max-turns:
        required: false
        type: number
        default: 15
    secrets:
      ANTHROPIC_API_KEY:
        required: true
```

```
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install -g @anthropic-ai/claude-code
      - env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          claude -p "$(cat ${ inputs.prompt-file })" \
            --allowed-tools "${ inputs.allowed-tools }" \
            --max-turns ${ inputs.max-turns } \
            --output-format json | jq -r '.result' > review.md
```

```
# 2. 호출하는 워크플로
# .github/workflows/pr.yml in any-team/repo
name: PR Workflow
on: pull_request
jobs:
  review:
    uses: org/.github/.github/workflows/claude-review.yml@main
    with:
      prompt-file: prompts/security.md
      allowed-tools: "Read,Grep,Bash(gh pr:*)"
    secrets:
      ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
```

reusable

workflow_call

재사용

inputs

인자

secrets

시크릿

Cross-repo

공유

GitLab CI / 기본 설정

.gitlab-ci.yml

```
# .gitlab-ci.yml
image: node:20
stages:
  - install
  - review
  - notify

variables:
  CLAUDE_VERSION: "1.2.0"

# Pipeline 캐싱 (node_modules)
cache:
  key: "claude-${CLAUDE_VERSION}"
  paths:
    - .npm-cache/

# Stage 1: 설치 (캐시 활용)
install:claude:
  stage: install
  script:
    - npm config set cache .npm-cache
    - npm install -g @anthropic-ai/claude-code@$CLAUDE_VERSION
    - claude --version # 확인
artifacts:
  paths:
    - .npm-cache/
```

.gitlab-ci.yml

```
# Stage 2: PR 리뷰
review:
  stage: review
  needs: [install:claude]
  only:
    - merge_requests
  variables:
    ANTHROPIC_API_KEY: $ANTHROPIC_API_KEY
  script:
    - !
      claude -p "MR !${CI_MERGE_REQUEST_IID} 리뷰: \
        $(git diff origin/${CI_MERGE_REQUEST_TARGET_BRANCH_NAME}...HEAD)" \
        --allowed-tools "Read,Grep,Glob" \
        --max-turns 15 \
        --output-format json | jq -r '.result' > review.md
  artifacts:
    paths:
      - review.md
    expire_in: 1 week
```

image

기본 환경

stages

단계

cache

재사용

artifacts

결과

GitLab CI / 고급 패턴

rules와 환경별 실행

GitLab rules

```
# rules: 조건부 실행 (only/except 대체)
review:
  stage: review
  rules:
    # MR이고 main 대상이고 .py 파일이 바뀐 경우만
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      changes:
        - "**/*.py"
        - "**/*.js"
      when: on_success

    # nightly 빌드는 모든 코드 분석
    - if: $CI_PIPELINE_SOURCE == "schedule"
      when: always

    # 그 외는 실행 안 함
    - when: never

script:
  - claude -p "..." --output-format json
```

```
# 환경별 설정
.review_base:
  script:
    - claude -p "$PROMPT" \
      --model "$MODEL" \
      --max-turns "$MAX_TURNS" \
      --output-format json | jq -r '.result'

review:staging:
  extends: .review_base
  environment: staging
  variables:
    PROMPT: "Staging PR 리뷰"
    MODEL: "claude-haiku-4-5"
    MAX_TURNS: "10"
  rules:
    - if: $CI_COMMIT_BRANCH == "staging"

review:production:
  extends: .review_base
  environment: production
  variables:
    PROMPT: "Production PR 종합 리뷰 + 보안 검토"
    MODEL: "claude-sonnet-4-5"
    MAX_TURNS: "30"
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

rules

조건부

extends

템플릿

environment

환경

changes

경로 필터

Jenkins / Jenkinsfile 기본

Declarative Pipeline

```
// Jenkinsfile
pipeline {
  agent {
    docker {
      image 'node:20-alpine'
      args '-v /var/lib/jenkins/.npm:/root/.npm'
    }
  }
  environment {
    ANTHROPIC_API_KEY = credentials('anthropic-api-key')
    CLAUDE_MAX_TURNS = '15'
  }
  options {
    timeout(time: 10, unit: 'MINUTES')
    ansiColor('xterm')
  }
}
```

```
stages {
  stage('Setup') {
    steps {
      sh 'npm install -g @anthropic-ai/claude-code'
      sh 'claude --version'
    }
  }
  stage('Review') {
    when {
      anyOf {
        branch 'main'
        changeRequest()
      }
    }
    steps {
      script {
        def review = sh(
          script: '''
            claude -p "PR 변경 사항 리뷰" \
              --max-turns $CLAUDE_MAX_TURNS \
              --output-format json | jq -r '.result'
          ''',
          returnStdout: true
        ).trim()
        writeFile file: 'review.md', text: review
      }
    }
  }
}
```

```
stage('Archive') {
  steps {
    archiveArtifacts artifacts: 'review.md',
      fingerprint: true
  }
}

post {
  success { echo '✓ Review completed' }
  failure { echo '✗ Review failed' }
  always { cleanWs() }
}
```

Jenkinsfile

docker

실행 환경

credentials

인증

when

조건

post

후처리

Jenkins / 병렬 실행

복수 분석 동시 수행

```
// 병렬 분석 패턴
pipeline {
  agent any
  environment {
    ANTHROPIC_API_KEY = credentials('anthropic-api-key')
  }
  stages {
    stage('Setup') {
      steps {
        sh 'npm install -g @anthropic-ai/claude-code'
      }
    }

    stage('Parallel Analysis') {
      parallel {
        stage('Security') {
          steps {
            sh '''
              claude -p "보안 관점 코드 분석" \
                --allowed-tools "Read,Grep,Glob" \
                --output-format json \
                > security.json
            '''
          }
        }

        stage('Performance') {
          steps {
            sh '''
              claude -p "성능 관점 분석" \
                --output-format json \
                > performance.json
            '''
          }
        }

        stage('Documentation') {
          steps {
            sh '''
              claude -p "문서화 검토" \
                --output-format json \
                > docs.json
            '''
          }
        }
      }
    }

    stage('Aggregate') {
      steps {
        sh '''
          jq -s '...' security.json performance.json \
            docs.json > final-report.json
        '''
        archiveArtifacts artifacts: 'final-report.json'
      }
    }
  }
}
```

parallel {}

동시 실행

3 stages

3 분석

JSON output

결과 파일

Aggregate

통합

Jenkins / Credentials

안전한 시크릿 관리

```
// 1. Jenkins UI에서 credentials 등록
// Manage Jenkins → Credentials → System → Global credentials
// Kind: Secret text
// ID: anthropic-api-key
// Secret: sk-ant-api03-...

// 2. environment 블록으로 주입
pipeline {
  environment {
    // Secret text 타입
    ANTHROPIC_API_KEY = credentials('anthropic-api-key')

    // Username/Password 타입 (자동 분할)
    // DB_CREDENTIALS_USR, DB_CREDENTIALS_PSW 자동 생성
    DB_CREDENTIALS = credentials('db-prod')

    // File 타입
    AWS_CONFIG = credentials('aws-config-file')
  }
}
```

```
// 3. withCredentials로 스코프 한정
pipeline {
  stages {
    stage('Sensitive') {
      steps {
        withCredentials([
          string(credentialsId: 'anthropic-api-key',
            variable: 'ANTHROPIC_API_KEY')
        ]) {
          sh '''
            ...
            claude -p "..." --output-format json
            ...
          '''
          // 블록 밖에서는 ANTHROPIC_API_KEY 노출 안 됨
        }
      }
    }
  }
}

// 4. AWS Bedrock 사용 시 (IAM Role)
pipeline {
  environment {
    CLAUDE_CODE_USE_BEDROCK = '1'
    AWS_REGION = 'us-east-1'
  }
  options {
    // EC2 instance role 자동 사용 또는
    // AWS Credentials 플러그인
    withAWS(role: 'arn:aws:iam::123:role/jenkins-claude') { ... }
  }
}
```

credentials

Secret text
기본

env
주입

withCredentials
스코프

AWS IAM
Bedrock

시크릿 관리 베스트 프랙티스

4가지 핵심 원칙

BP 1

OIDC 우선

AWS Bedrock는 OIDC로
임시 자격증명.
API key 저장 회피.

BP 2

최소 권한

CI용 API key는 별도 발급,
사용량 한도 설정, 정기 회전.

BP 3

환경별 분리

staging과 production에
다른 key.
환경 간 오염 차단.

BP 4

감사 로깅

모든 CI 호출을 SIEM에 기록.
비정상 사용 즉시 감지.

NEVER

시크릿을 코드, 워크플로 YAML, 로그, Slack에 평문으로 저장하지 마세요. PR diff에 노출되면 즉시 회전하고 사고 보고.

OIDC

권장

Least-priv

최소

Separation

분리

Audit

감사

CI에서 비용 통제

예산 초과 방지

cost control

```
# 1. --max-turns로 도구 호출 제한
$ claude -p "..." --max-turns 10
# 10번 호출 후 강제 종료
# 2. --max-tokens로 응답 토큰 제한
$ claude -p "..." --max-tokens 2000
# 응답이 2000 토큰 초과 시 잘림
# 3. 모델 선택 (Haiku 활용)
# CI의 단순 작업은 Haiku로 충분, 10x 절감
# 복잡 분석만 Sonnet, 핵심 결정만 Opus
# 4. timeout으로 시간 제한
$ timeout 300 claude -p "..." # 5분 제한
# Unix timeout 명령으로 강제 종료
# 5. 일일 사용량 모니터링
# GitHub Actions에서:
- name: Check daily usage
  run: |
    USAGE=$(curl -H "x-api-key: $API_KEY" \
      https://api.anthropic.com/v1/usage)
    SPENT=$(echo "$USAGE" | jq -r '.daily_spent_usd')
    if [ "$(echo "$SPENT" > 100 | bc)" = "1" ]; then
      echo "❌ 일일 한도 초과 ($$SPENT)" >&2
      exit 1
    fi
```

```
# 6. PR 라벨로 분기 (작은 PR만 자동 리뷰)
- name: Check PR size
  run: |
    CHANGES=$(gh pr view $PR --json additions,deletions \
      | jq '.additions + .deletions')
    if [ "$CHANGES" -gt 1000 ]; then
      echo "PR 너무 큼 ($CHANGES줄), 수동 리뷰 필요"
      exit 0 # 정상 종료, 리뷰 스킵
    fi
```

```
# 7. 캐싱으로 재호출 방지
# 같은 입력은 결과 캐싱
# Redis나 GitHub Actions cache 활용
```

Limits

강제 한도

Cheap model

저비용

Monitor

모니터

Skip large

큰 작업 제외

CI 보안 검증

악성 PR 방어

```
# 위험: 외부 기여자가 악성 PR을 만들어 시크릿을 탈취하려 함
# 1. fork PR은 secrets 차단 (GitHub 기본)
# pull_request 이벤트에서 PR이 fork에서 오면
# secrets는 자동으로 비어 있음
# 의도: 악성 PR이 시크릿을 사용 못 함
# 2. workflow_run + pull_request_target 패턴
# Step A: 안전한 워크플로 (코드 다운로드만)
name: PR Trigger
on: pull_request
jobs:
  collect:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/upload-artifact@v4
        with:
          name: pr-data
          path: |
            .
```

```
# Step B: 권한 있는 워크플로
name: Run Claude
on:
  workflow_run:
    workflows: ["PR Trigger"]
    types: [completed]
permissions:
  pull-requests: write
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: pr-data
      - env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          claude -p "... " \
            --disallowed-tools "Write,Edit,Bash" \
            --output-format json

# 3. 도구 권한을 좁게 제한
$ claude -p "... " \
  --allowed-tools "Read,Grep,Glob" \
  --disallowed-tools "Write,Edit,Bash"
# 코드 변경은 금지
```

security

Fork block

fork PR

workflow_run

분리

Read-only

읽기만

Defense

방어

CI 롤백 전략

AI 자동 변경의 안전망

STRATEGY 1

PR 우선

AI가 main에 직접 push 금지.
항상 PR로 생성 후 사람 승인.

STRATEGY 2

Feature Flag

AI 변경은 feature flag 뒤에.
문제 시 즉시 비활성화 가능.

STRATEGY 3

점진 적용

canary 1% → 10% →
50% → 100%. 단계별 메
트릭 검증.

STRATEGY 4

자동 롤백

에러율/지연 임계치 초과 시
이전 버전으로 자동 복원.

GOLDEN RULE

AI 자동화의 모든 변경은 즉시 되돌릴 수 있어야 하며, Revert가 어려운 변경은 사람의 승인 단계를 추가.

PR-only

리뷰

Flag

토글

Canary

점진

Auto-revert

자동

CI 모니터링과 알람

실패 시 즉시 인지

monitoring

```
# 1. Slack 알람 (성공/실패)
- name: Notify Slack
  if: always()
  uses: 8398a7/action-slack@v3
  with:
    status: ${ job.status }
    channel: '#ci-claude'
    fields: repo,branch,commit,author,workflow
```

```
# 2. CloudWatch 메트릭 전송
- name: Send metrics
  if: always()
  env:
    AWS_REGION: us-east-1
  run: |
    DURATION=$(jq '.duration_ms' result.json)
    TOKENS=$(jq '.usage.output_tokens' result.json)
    aws cloudwatch put-metric-data \
      --namespace ClaudeCI \
      --metric-data \
        MetricName=Duration,Value=$DURATION \
        MetricName=Tokens,Value=$TOKENS
```

```
# 3. Datadog 전송
curl -X POST "https://api.datadoghq.com/api/v1/events" \
  -H "DD-API-KEY: $DD_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Claude CI Result",
    "text": "PR #123 review: 5 issues found",
    "tags": ["service:ci", "result:warning"]
  }'
```

```
# 4. PagerDuty (심각도 높은 실패)
if [ "$EXIT_CODE" -ne 0 ] && [ "$BRANCH" = "main" ]; then
  curl -X POST "https://events.pagerduty.com/v2/enqueue" \
    -H "Content-Type: application/json" \
    -d "$(jq -nc \
      --arg key "$PD_KEY" \
      --arg msg "Claude CI failed on main" \
      '{routing_key: $key, event_action: "trigger", \
        payload: {summary: $msg, severity: "error"}}')'"
fi
```

```
# 5. 메트릭 대시보드
# - Grafana
# - Datadog Dashboard
# - CloudWatch Dashboard
```

Slack
일반 알람

CloudWatch
메트릭

Datadog
이벤트

PagerDuty
심각

PR 자동 처리 완성 예시

GitHub Actions 전체 흐름

```
# .github/workflows/pr-complete.yml
name: PR Complete Workflow
on:
  pull_request:
    types: [opened, synchronize]
permissions:
  contents: read
  pull-requests: write
  issues: write
jobs:
  size-check:
    runs-on: ubuntu-latest
    outputs:
      should_review: ${ steps.check.outputs.should_review }
    steps:
      - id: check
        env:
          GH_TOKEN: ${ github.token }
        run: |
          CHANGES=$(gh pr view ${ github.event.pull_request.number } \
            --json additions,deletions -q '.additions + .deletions')
          if [ "$CHANGES" -gt 2000 ]; then
            echo "should_review=false" >> $GITHUB_OUTPUT
          else
            echo "should_review=true" >> $GITHUB_OUTPUT
          fi
```

```
review:
  needs: size-check
  if: needs.size-check.outputs.should_review == 'true'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with: { fetch-depth: 0 }
    - uses: actions/setup-node@v4
      with: { node-version: '20' }
    - run: npm install -g @anthropic-ai/claude-code
    - name: Review
      id: review
      env:
        ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
      run: |
        claude -p "PR 종합 리뷰" \
          --allowed-tools "Read,Grep,Glob,Bash(gh pr:*)" \
          --max-turns 15 \
          --output-format json > review.json
    - name: Post comment
      uses: marocchino/sticky-pull-request-comment@v2
      with:
        message-path: ${ steps.review.outputs.message-path }
    - name: Notify on failure
      if: failure()
      uses: 8398a7/action-slack@v3
      with:
        status: failure
        channel: '#ci-alerts'
```

complete example

Size check

크기 필터

Review

분석

Comment

게시

Notify

알림

다른 CI 시스템

CircleCI, Travis

```
# 1. CircleCI (.circleci/config.yml)
version: 2.1
jobs:
  review:
    docker:
      - image: node:20-alpine
    steps:
      - checkout
      - run: npm install -g @anthropic-ai/claude-code
      - run:
          name: Run Claude
          environment:
            ANTHROPIC_API_KEY: $ANTHROPIC_API_KEY
          command: |
            claude -p "..." --output-format json

workflows:
  pr-review:
    jobs:
      - review:
          filters:
            branches:
              ignore: main
```

```
# 2. Travis CI (.travis.yml)
language: node_js
node_js: '20'
before_install:
  - npm install -g @anthropic-ai/claude-code
env:
  - ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY
script:
  - claude -p "..." --output-format json > result.json
```

other CI

CircleCI
config.yml

Travis
.travis.yml

Azure
pipelines

Universal
공통 패턴

CI 통합 베스트 프랙티스 종합

8가지 핵심 체크

No	항목	확인
1	시크릿은 CI의 secret 기능으로만	YAML 평문 금지
2	--max-turns로 도구 호출 제한	폭주 방지
3	--allowed-tools로 권한 최소화	Read만 허용 가능
4	CI 전용 API key (분리)	한도 설정
5	fork PR은 secrets 차단	기본 동작
6	큰 PR 자동 스킵 (라인 수 체크)	비용 통제
7	실패 시 Slack/PagerDuty 알림	즉시 인지
8	메트릭 수집과 대시보드	지속 모니터링

8 items
체크리스트

Layered
계층 방어

Mandatory
필수

Quarterly
주기 검토

Part 4 Recap

CI/CD 통합 핵심 정리

TAKEAWAY 1

3대 CI 플랫폼

GitHub Actions, GitLab
CI, Jenkins 통합 패턴 모두
마스터

TAKEAWAY 2

공통 패턴

Install → Auth (Secret)
→ Headless 호출 → JSON
파싱

TAKEAWAY 3

보안과 비용

OIDC, fork 방어, max-
turns, 모델 적합, 큰 PR 스
킵

TAKEAWAY 4

운영

PR 게시, Slack 알림, 메트
릭 수집, 자동 롤백

NEXT - PART 5

Part 5는 자동화 패턴입니다. 실무 시나리오 5가지를 깊이 있게 다룹니다. PR 리뷰 봇, 이슈 트리아지, 코드 마이그레이션, 보안 감사, 매일 보고서.

Part 4

완료

Part 5

패턴 (20p)

Part 6-7

Env+Debug (25p)

~2h

잔여

CHAPTER 05 – PART 05

CLI Reference – 자동화

전체 CLI 플래그, Headless 모드, CI/CD 자동화



Part 5 / 자동화 패턴

실무 시나리오 5가지 깊이 있게

PATTERN 1

PR 리뷰 봇

4장에 걸쳐 완전 자동화 PR 리뷰 봇 구현 - 코멘트, 라벨, 승인

PATTERN 2

이슈 트리아지

4장에 걸쳐 새 이슈 자동 분류 - 카테고리, 우선순위, 담당자

PATTERN 3

코드 마이그레이션

3장에 걸쳐 대규모 코드베이스 일괄 변환 - 안전 가이드라인

PATTERN 4,5

보안감사/ 보고서

Pattern 4 보안 감사는 정기 보안 스캔 자동화. Pattern 5 매일 보고서는 팀 활동을 자동 요약해 Slack 게시.

20 Slides

Part 5 분량

~40 min

예상 강의 시간

5 patterns

완성 코드

Copy-paste

즉시 활용

Pattern 1 / PR 리뷰 봇 설계

동작 흐름 6단계

STEP 01

PR 이벤트
트리거

STEP 02

PR 정보
수집

STEP 03

AI 분석

STEP 04

코멘트
게시

STEP 05

라벨
부착

STEP 06

결과
알림

DESIGN GOAL

사람이 하는 PR 리뷰의 첫 패스를 완전 자동화합니다. AI는 코드 품질, 보안, 테스트, 문서 4가지 관점을 검토하고 사람은 비즈니스 결정에 집중합니다.

관점

코드 품질

보안

테스트

Claude의 역할

자동 검토

패턴 감지

커버리지 체크

사람의 역할

복잡한 트레이드오프

비즈니스 영향 판단

엣지 케이스 설계

6 steps

흐름

4 관점

검토

Comment

게시

Label

분류

Pattern 1 / 핵심 스크립트

pr-review-bot.sh

```
#!/bin/bash
# pr-review-bot.sh
set -euo pipefail

PR_NUM="${1:?Usage: pr-review-bot.sh <PR-number>}"

# 1. PR 정보 수집
PR_DATA=$(gh pr view "$PR_NUM" \
  --json title,body,additions,deletions,changedFiles,author)
TITLE=$(echo "$PR_DATA" | jq -r '.title')
ADDITIONS=$(echo "$PR_DATA" | jq -r '.additions')
DELETIONS=$(echo "$PR_DATA" | jq -r '.deletions')

# 2. 큰 PR 차단 (비용 통제)
TOTAL_CHANGES=$((ADDITIONS + DELETIONS))
if [ "$TOTAL_CHANGES" -gt 2000 ]; then
  gh pr comment "$PR_NUM" --body \
    "⚠️ PR이 너무 큼니다 ($TOTAL_CHANGES 줄). 분할 요청."
  exit 0
fi
```

```
# 3. AI 분석 (구조화 JSON 응답)
REVIEW=$(claude -p "$(cat <<EOF
다음 PR을 4가지 관점에서 분석. 출력은 JSON.
```

pr-review-bot.sh

```
PR: $TITLE
Diff:
$(gh pr diff "$PR_NUM")

{
  "verdict": "approve|comment|request_changes",
  "labels": ["bugfix","feature","refactor","docs","test"],
  "severity": "low|medium|high",
  "comments": [
    { "file": "...", "line": 0, "severity": "...", "msg": "..." }
  ],
  "summary": "한 줄 요약"
}
EOF
)" \
  --allowed-tools "Read,Grep,Glob,Bash(gh pr:*)" \
  --max-turns 15 \
  --output-format json | jq -r '.result' | jq .)

echo "$REVIEW" > "/tmp/pr-$PR_NUM-review.json"
```

gh CLI
PR 정보

Size guard
비용

Structured
JSON 응답

fromjson
파싱

Pattern 1 / 결과 처리

코멘트, 라벨, 승인 자동화

```
# 이어서: pr-review-bot.sh
```

```
# 4. 라벨 부착
```

```
LABELS=$(echo "$REVIEW" | jq -r '.labels | join(",")')
gh pr edit "$PR_NUM" --add-label "$LABELS"
```

```
# 5. 종합 코멘트 게시
```

```
SUMMARY=$(echo "$REVIEW" | jq -r '.summary')
SEVERITY=$(echo "$REVIEW" | jq -r '.severity')
VERDICT=$(echo "$REVIEW" | jq -r '.verdict')
```

```
EMOJI="✅"
```

```
[ "$VERDICT" = "comment" ] && EMOJI="💬"
```

```
[ "$VERDICT" = "request_changes" ] && EMOJI="🚫"
```

```
COMMENT_BODY=$(cat <<EOF
```

```
## $EMOJI Claude AI 리뷰
```

```
**Summary:** $SUMMARY
```

```
**Severity:** $SEVERITY
```

```
**Verdict:** $VERDICT
```

```
### 세부 코멘트
```

```
$(echo "$REVIEW" | jq -r '.comments[] |
  "- **\(.file):\(.line)** [\(.severity)] \(.msg)'"')
---
```

```
*자동 생성. 사람의 최종 리뷰가 필요합니다.*
```

```
EOF
```

```
)
```

```
gh pr comment "$PR_NUM" --body "$COMMENT_BODY"
```

```
# 6. 인라인 코멘트 (각 파일별)
```

```
echo "$REVIEW" | jq -c '.comments[]' | while read -r c; do
  FILE=$(echo "$c" | jq -r '.file')
  LINE=$(echo "$c" | jq -r '.line')
  MSG=$(echo "$c" | jq -r '.msg')
```

```
if [ "$LINE" -gt 0 ]; then
```

```
  gh api "/repos/{owner}/{repo}/pulls/$PR_NUM/comments" \
    -X POST \
    -f body="$MSG" \
    -f path="$FILE" \
    -f line="$LINE" \
    -f commit_id="$(gh pr view $PR_NUM --json headRefOid \
      -q '.headRefOid')"
```

```
fi
```

```
done
```

```
echo "✓ PR #$PR_NUM 리뷰 완료"
```

pr-review-bot.sh (cont)

Labels

자동 분류

Summary

종합

Inline

라인별

gh api

API 호출

Pattern 1 / GitHub Actions 통합

워크플로 등록

```
# .github/workflows/pr-review-bot.yml
name: Claude PR Review Bot
on:
  pull_request:
    types: [opened, synchronize]
permissions:
  pull-requests: write
  issues: write
  contents: read
jobs:
  review:
    runs-on: ubuntu-latest
    timeout-minutes: 15
```

```
steps:
  - uses: actions/checkout@v4
    with:
      fetch-depth: 0
  - uses: actions/setup-node@v4
    with:
      node-version: '20'
  - name: Install Claude
    run: npm install -g @anthropic-ai/claude-code
  - name: Run review bot
    env:
      ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
      GH_TOKEN: ${ github.token }
    run: |
      chmod +x .github/scripts/pr-review-bot.sh
      .github/scripts/pr-review-bot.sh \
        ${ github.event.pull_request.number }
  - name: Notify on failure
    if: failure()
    uses: 8398a7/action-slack@v3
    with:
      status: failure
      channel: '#ci-alerts'
      text: 'PR review bot failed'

# 효과
# - 모든 새 PR에 자동 리뷰
# - 라벨 자동 부착으로 분류
# - 인라인 코멘트로 정확한 위치 표시
# - 사람 리뷰는 비즈니스 결정에 집중
```

pr-review-bot.yml

on PR
트리거

permissions
권한

env vars
인증

failure alert
알림

Pattern 2 / 이슈 트리아지 설계

새 이슈 자동 분류

STEP 01

새 이슈
생성

STEP 02

내용
분석

STEP 03

카테고리
분류

STEP 04

우선순위
결정

STEP 05

담당자
할당

STEP 06

슬랙
알림

PROBLEM

하루 수십 개의 이슈가 들어오면 트리아지에만 많은 시간이 듭니다. AI가 1차 분류와 우선순위를 자동 결정하고 사람은 검토만 합니다.

이슈 자동 분류 카테고리 예시

- bug: 버그 보고
- feature-request: 기능 요청
- documentation: 문서 개선

- question: 질문
- security: 보안 관련
- performance: 성능 이슈
- enhancement: 개선 제안

categories

6 steps

흐름

Categories

분류

Priority

우선순위

Assignee

담당자

Pattern 2 / 분류 스크립트

triage-issue.sh

```
#!/bin/bash
# triage-issue.sh
set -euo pipefail

ISSUE_NUM="${1:?Usage: triage-issue.sh <issue-number>}"

# 1. 이슈 정보 수집
ISSUE=$(gh issue view "$ISSUE_NUM" --json title,body,author,createdAt)

# 2. AI 분석 (구조화 JSON)
ANALYSIS=$(claude -p "$(cat <<EOF
다음 이슈를 분석하고 JSON으로 응답:

이슈 제목: $(echo "$ISSUE" | jq -r '.title')
내용:
$(echo "$ISSUE" | jq -r '.body')

응답 형식:
{
  "category": "bug|feature-request|documentation|question|security|performance|enhancement",
  "priority": "critical|high|medium|low",
  "estimated_effort": "trivial|small|medium|large",
  "suggested_assignees": ["github-username"],
  "needs_more_info": true|false,
  "duplicate_of": null,
  "summary": "한 문장 요약"
}
```

```
판단 기준:
- security: 보안 취약점, 데이터 노출
- critical: 프로덕션 중단, 데이터 손실
- high: 주요 기능 동작 불가
EOF
)" \
  --allowed-tools "Read,Grep,Glob,Bash(gh issue:*)" \
  --max-turns 10 \
  --output-format json | jq -r '.result' | jq .)

echo "$ANALYSIS" > "/tmp/issue-$ISSUE_NUM.json"

# 3. 결과 추출
CATEGORY=$(echo "$ANALYSIS" | jq -r '.category')
PRIORITY=$(echo "$ANALYSIS" | jq -r '.priority')
EFFORT=$(echo "$ANALYSIS" | jq -r '.estimated_effort')
NEEDS_INFO=$(echo "$ANALYSIS" | jq -r '.needs_more_info')
SUMMARY=$(echo "$ANALYSIS" | jq -r '.summary')

echo "[ $CATEGORY ] [ $PRIORITY ] [ $EFFORT ] $SUMMARY"
```

triage-issue.sh

gh CLI
이슈 정보

Structured
JSON 응답

Criteria
판단 기준

Extract
값 추출

Pattern 2 / 액션 실행

라벨, 담당자, 알림

```
# 이어서: triage-issue.sh
```

```
# 4. 라벨 부착
```

```
gh issue edit "$ISSUE_NUM" \  
  --add-label "$CATEGORY,priority:$PRIORITY,effort:$EFFORT"
```

```
# 5. 정보 부족 시 자동 응답
```

```
if [ "$NEEDS_INFO" = "true" ]; then  
  gh issue comment "$ISSUE_NUM" --body "$(cat <<EOF  
  🙋 이슈 등록 감사합니다.
```

좀 더 정확히 도와드리기 위해 다음 정보가 필요합니다:

- 발생 환경 (OS, 브라우저, 버전)
- 재현 단계
- 예상 동작 vs 실제 동작
- 스크린샷 또는 로그

```
업데이트 부탁드립니다.
```

```
EOF  
)"  
  gh issue edit "$ISSUE_NUM" --add-label "needs-info"  
fi
```

```
# 6. 보안 이슈는 별도 채널로
```

```
if [ "$CATEGORY" = "security" ]; then  
  curl -X POST "$SECURITY_SLACK_WEBHOOK" \  
    -H 'Content-Type: application/json' \  
    -d "$(jq -n --arg num "$ISSUE_NUM" --arg sum "$SUMMARY" \  
      '{text: ":warning: Security issue #\($num): \($sum)'}")"  
fi
```

```
# 7. 담당자 자동 할당 (suggested의 첫 번째)
```

```
ASSIGNEE=$(echo "$ANALYSIS" | jq -r '.suggested_assignees[0] // empty')  
if [ -n "$ASSIGNEE" ] && [ "$ASSIGNEE" != "null" ]; then  
  gh issue edit "$ISSUE_NUM" --add-assignee "$ASSIGNEE"  
fi
```

```
# 8. 중복 감지 시 안내
```

```
DUP=$(echo "$ANALYSIS" | jq -r '.duplicate_of // empty')  
if [ -n "$DUP" ] && [ "$DUP" != "null" ]; then  
  gh issue comment "$ISSUE_NUM" --body \  
    "🔍 이 이슈는 #DUP와 중복될 수 있습니다."  
fi
```

```
echo "📄 Issue #ISSUE_NUM 트리아지 완료: $CATEGORY/$PRIORITY"
```

triage actions

Labels

분류

Needs info

자동 응답

Security

별도 알림

Assignee

할당

Pattern 2 / GitHub Actions 통합

issue.opened 트리거

```
# .github/workflows/triage-issue.yml
name: Auto-triage Issues
```

```
on:
  issues:
    types: [opened, reopened]
```

```
permissions:
  issues: write
  contents: read
```

```
jobs:
  triage:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm install -g @anthropic-ai/claude-code

      - name: Triage
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
          GH_TOKEN: ${ github.token }
          SECURITY_SLACK_WEBHOOK: ${ secrets.SECURITY_WEBHOOK }
        run: |
          chmod +x .github/scripts/triage-issue.sh
          .github/scripts/triage-issue.sh \
            ${ github.event.issue.number }
```

```
# 통합 활용
# - 매일 100개 이슈가 들어와도 즉시 분류
# - 보안 이슈는 별도 채널로 5분 내 알림
# - 사람은 검토와 우선순위 조정에 집중

# 메트릭 (3개월 운영 후 예시)
# - 평균 분류 시간: 사람 8분 → AI 30초
# - 잘못된 라벨 비율: 사람 12% → AI 8%
# - 응답 시간: 평균 12시간 → 즉시
# - 사람이 트리아지에 쓰는 시간: 70% 감소
```

triage-issue.yml

issues

트리거

Multi env

시크릿

70% saved

시간

5 min

응답

Pattern 3 / 코드 마이그레이션 설계

대규모 일괄 변환

USE CASE

시나리오 예시

Vue 2 → Vue 3 마이그레이션,
jQuery → React, JS → TS, lodash
제거 같은 대규모 변환

CHALLENGE

도전 과제

수백 파일을 동시에 변환, 일관성 유지,
안전한 검증, 부분 실패 복구

APPROACH

AI + 사람

AI가 변환 수행, 자동 테스트로 검증, 사
람은 의심스러운 결과만 검토

안전한 마이그레이션 흐름

1. 변환 규칙 정의 (한 번)
2. 작은 샘플 (5-10 파일)로 검증
3. 결과 확인 후 본격 변환
4. 각 파일 변환 후 즉시 테스트

5. 실패 시 자동 롤백 + 사람 검토
6. 전체 완료 후 통합 테스트

safe migration

Batch

대규모

Consistent

일관성

Verified

검증

Rollback

복구

Pattern 3 / 변환 스크립트

migrate-files.sh

```
#!/bin/bash
# migrate-files.sh - 대규모 코드 마이그레이션
set -euo pipefail
PATTERN="${1:?Usage: migrate-files.sh <glob> <rules-file>}"
RULES_FILE="${2:?}"
mkdir -p .migration/{backup,failed,success}
# 변환 규칙 로드
RULES=$(cat "$RULES_FILE")
# 파일 목록
FILES=$(find . -type f -name "$PATTERN" \
  -not -path ".node_modules/*" \
  -not -path ".migration/*")
TOTAL=$(echo "$FILES" | wc -l)
COUNT=0
echo "총 $TOTAL개 파일 마이그레이션 시작"
# 각 파일 변환
for file in $FILES; do
  COUNT=$((COUNT + 1))
  echo "[${COUNT}/${TOTAL}] $file"
  # 백업
  cp "$file" ".migration/backup/$(echo "$file" | tr '/' '_')"
  # AI 변환
  ORIGINAL=$(cat "$file")
  NEW=$(claude -p "$(cat <<EOF
```

다음 코드를 변환 규칙에 따라 마이그레이션:

변환 규칙:

\$RULES

원본 코드 (\$file):

\$ORIGINAL

변환된 코드만 출력 (설명 없이).

EOF

)" \

--allowed-tools "Read" \

--max-turns 3 \

--output-format json | jq -r '.result')

결과 저장

echo "\$NEW" > "\$file"

즉시 테스트

if npm test --silent -- "\$file" 2>/dev/null; then

cp "\$file" ".migration/success/\$(echo "\$file" | tr '/' '_')"

echo " ✓ 통과"

else

실패 시 원본 복원

cp ".migration/backup/\$(echo "\$file" | tr '/' '_')" "\$file"

cp "\$file.failed" ".migration/failed/" 2>/dev/null || true

echo " x 실패 (롤백 완료)"

fi

done

통계

SUCCESS=\$(ls .migration/success | wc -l)

FAILED=\$(ls .migration/failed | wc -l)

echo "완료: \$SUCCESS 성공, \$FAILED 실패"

migrate-files.sh

Backup

안전망

Per-file test

즉시 검증

Auto rollback

자동 복구

Stats

통계

Pattern 3 / 실패 분석

롤백된 파일 자동 진단

```
#!/bin/bash
# analyze-failures.sh - 마이그레이션 실패 파일 분석
mkdir -p .migration/analysis
# 각 실패 파일 분석
for backup in .migration/failed/*; do
    ORIG_NAME=$(basename "$backup" | tr '_' '/')
    OUT=".migration/analysis/${basename "$backup"}.failed.md"
    cLaude -p "$(cat <<EOF
다음 코드 변환이 테스트 실패했습니다. 이유를 분석:
원본 파일 위치: $ORIG_NAME
변환 시도 코드:
$(cat "$backup")
분석 요청:
1. 변환 실패 원인 (테스트 에러 메시지 포함)
2. 어느 부분이 문제인가
3. 수정 방법 제안 (3가지 옵션)
4. 사람의 검토가 필요한 이유
5. 다음 시도 시 변환 규칙에 추가할 예외
출력 형식: Markdown 보고서
EOF
)" \
    --allowed-tools "Read,Bash(npm test:*)" \
    --max-turns 10 \
    --output-format json | jq -r '.result' > "$OUT"
    echo "✓ 분석 완료: $OUT"
done
```

```
# 통합 보고서
{
    echo "# 마이그레이션 실패 분석 보고서"
    echo "생성: $(date)"
    echo ""
    echo "## 통계"
    echo "- 실패 파일: $(ls .migration/failed | wc -l)개"
    echo "- 성공 파일: $(ls .migration/success | wc -l)개"
    echo ""
    echo "## 파일별 분석"
    for f in .migration/analysis/*.md; do
        echo ""
        echo "### $(basename "$f" .md)"
        cat "$f"
        echo ""
        echo "---"
    done
} > .migration/REPORT.md

echo "✓ 종합 보고서: .migration/REPORT.md"
```

analyze-failures.sh

Per-file
개별 분석

Test logs
에러 추적

Suggestions
수정 제안

Report
통합

Pattern 4 / 보안 감사 설계

정기 보안 스캔 자동화

STEP 01

스케줄
트리거

STEP 02

코드
스캔

STEP 03

취약점
분류

STEP 04

CVE
조회

STEP 05

보고서
생성

STEP 06

게시
+ 알림

SCOPE

주간 또는 월간 정기 실행으로 코드베이스 전체를 보안 관점에서 스캔.

새로 추가된 취약점을 빠르게 발견하고 우선순위에 따라 대응.

검사 항목

의존성 취약점

코드 취약점

비밀 노출

도구

npm audit, pip-audit

semgrep, sonarqube

gitleaks, trufflehog

Claude의 역할

결과 분석과 우선순위

오탐 필터링

심각도 평가

Schedule

정기

Multi-tool

다중 도구

AI analyze

분석

Priority

우선순위

Pattern 4 / 보안 감사 스크립트

security-audit.sh

```
#!/bin/bash
# security-audit.sh - 정기 보안 감사
set -euo pipefail
REPORT_DIR="reports/security-$(date +%Y-%m-%d)"
mkdir -p "$REPORT_DIR"
# 1. 의존성 스캔
echo "[1/4] 의존성 스캔..."
npm audit --json > "$REPORT_DIR/npm-audit.json" || true
# 2. 코드 스캔 (semgrep)
echo "[2/4] 코드 스캔..."
semgrep --config=auto --json -o "$REPORT_DIR/semgrep.json" . || true
# 3. 비밀 노출 검사
echo "[3/4] 비밀 노출 검사..."
gitleaks detect --report-path "$REPORT_DIR/gitleaks.json" \
  --no-banner -f json || true
```

```
# 4. AI 통합 분석
echo "[4/4] AI 통합 분석..."
claude -p "$(cat <EOF
다음 보안 스캔 결과를 통합 분석:
npm audit:
$(cat "$REPORT_DIR/npm-audit.json" | head -200)
semgrep:
$(cat "$REPORT_DIR/semgrep.json" | head -200)
gitleaks:
$(cat "$REPORT_DIR/gitleaks.json" 2>/dev/null | head -100)
응답 형식 (JSON):
{
  "critical_count": 0,
  "high_count": 0,
  "medium_count": 0,
  "low_count": 0,
  "false_positive_likely": [],
  "top_priorities": [
    {
      "rank": 1, "issue": "...", "severity": "...",
      "location": "...", "fix": "..."
    }
  ],
  "executive_summary": "한 단락 요약"
}
EOF
)" \
  --allowed-tools "Read,Grep" \
  --max-turns 20 \
  --output-format json | jq -r '.result' | jq . \
  > "$REPORT_DIR/ai-analysis.json"
```

```
# 5. 마크다운 보고서 생성
ANALYSIS=$(cat "$REPORT_DIR/ai-analysis.json")
SUMMARY=$(echo "$ANALYSIS" | jq -r '.executive_summary')
security-audit.sh
{
  echo "# 보안 감사 보고서 - $(date +%Y-%m-%d)"
  echo ""
  echo "## 요약"
  echo "$SUMMARY"
  echo ""
  echo "## 우선순위 Top 5"
  echo "$ANALYSIS" | jq -r '.top_priorities[] |
    ## \(.rank). [\(.severity)] \(.issue)\n위치: \(.location)\n수정: \(.fix)\n"'
} > "$REPORT_DIR/REPORT.md"

echo "\n 보고서: $REPORT_DIR/REPORT.md"
```

3 scanners

도구 통합

AI synthesis

종합

Top 5

우선순위

Report

게시 가능

Pattern 4 / 정기 실행

GitHub Actions schedule

```
# .github/workflows/security-audit.yml
name: Weekly Security Audit
on:
  schedule:
    - cron: '0 9 * * 1' # 매주 월요일 오전 9시 UTC
  workflow_dispatch: # 수동 트리거도 가능
permissions:
  contents: read
  issues: write
  pull-requests: write
jobs:
  audit:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # gitleaks는 전체 이력 필요
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - name: Install tools
        run: |
          npm install -g @anthropic-ai/claude-code
          python3 -m pip install semgrep
          curl -sSfL https://raw.githubusercontent.com/gitleaks/\
            gitleaks/master/install.sh | sh -s -- -b /usr/local/bin
```

```
- name: Run audit
  env:
    ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
  run: |
    chmod +x .github/scripts/security-audit.sh
    .github/scripts/security-audit.sh

- name: Upload report
  uses: actions/upload-artifact@v4
  with:
    name: security-audit-${ github.run_id }
    path: reports/security-*

- name: Create issue if critical
  env:
    GH_TOKEN: ${ github.token }
  run: |
    CRITICAL=$(jq '.critical_count' reports/*/ai-analysis.json)
    if [ "$CRITICAL" -gt 0 ]; then
      gh issue create \
        --title "🔴 보안 감사: $CRITICAL Critical 발견" \
        --body-file reports/*/REPORT.md \
        --label "security,critical"
    fi
```

security-audit.yml

Weekly cron

정기

Multi-tool

도구 설치

Artifact

결과 저장

Auto issue

심각 시

Pattern 5 / 매일 보고서 설계

팀 활동 자동 요약

STEP 01

오전 9시
트리거

STEP 02

어제 활동
수집

STEP 03

카테고리
분류

STEP 04

AI
요약

STEP 05

Markdown
포맷

STEP 06

Slack
게시

BENEFIT

팀 stand-up 회의 전에 어제 활동을 자동 요약해 공유. 사람은 자신이 한 일을 다시 적을 필요 없이 이미 정리된 내용을 검토만 함.

수집되는 데이터 6가지

1. 머지된 PR 목록과 카테고리
2. 클로즈된 이슈와 해결 방법
3. 새로 열린 이슈와 우선순위
4. 활발한 contributor 통계

5. 빌드/배포 상태
6. 모니터링 알람 발생 여부

data sources

Daily 9 AM

시각

6 sources

데이터

AI summary

요약

Slack

채널

Pattern 5 / 보고서 스크립트

daily-report.sh 완성

```
#!/bin/bash
# daily-report.sh
set -euo pipefail
DATE=$(date -d "yesterday" +%Y-%m-%d)
WORKDIR="/tmp/daily-$DATE"
mkdir -p "$WORKDIR"

# 1. 데이터 수집
gh pr list --state merged \
  --search "merged:$DATE" \
  --json title,author,labels,additions,deletions \
  > "$WORKDIR/prs.json"
gh issue list --state closed \
  --search "closed:$DATE" \
  --json title,labels,author > "$WORKDIR/closed-issues.json"
gh issue list --state open \
  --search "created:$DATE" \
  --json title,labels,author > "$WORKDIR/new-issues.json"
```

```
# 2. AI 요약 (구조화)
REPORT=$(claude -p "$(cat <<EOF
어제($DATE) 팀 활동을 stand-up 보고서로 요약:

머지된 PR:
$(cat $WORKDIR/prs.json | jq -c '.')

클로즈된 이슈:
$(cat $WORKDIR/closed-issues.json | jq -c '.')

새 이슈:
$(cat $WORKDIR/new-issues.json | jq -c '.')

응답: 다음 구조의 Markdown
# 📅 Daily Report - $DATE

## 🌟 Highlights (3-5개)
- ...

## ✅ 완료된 작업
- 카테고리별 정리

## 📢 새로운 이슈
- 우선순위별 정리

## 👥 Active Contributors
- top 5
```

```
## ⚠️ 주목할 사항
- 긴급한 것, 차단된 것
EOF
)" \
  --model claude-sonnet-4-5 \
  --max-turns 5 \
  --output-format json | jq -r '.result')

echo "$REPORT" > "$WORKDIR/report.md"

# 3. Slack 게시 (Block Kit)
PAYLOAD=$(jq -n --rawfile r "$WORKDIR/report.md" '{
  blocks: [
    { type: "section",
      text: { type: "mrkdwn", text: $r } }
  ]
}')
```

```
curl -X POST "$SLACK_WEBHOOK" \
  -H 'Content-Type: application/json' \
  -d "$PAYLOAD"

echo "📩 매일 보고서 게시 완료"
```

daily-report.sh

Multi-source

수집

Structured MD

포맷

Block Kit

Slack

rawfile

안전 주입

Pattern 5 / 스케줄 등록

GitHub Actions cron

```
# .github/workflows/daily-report.yml
name: Daily Team Report
on:
  schedule:
    - cron: '0 0 * * 1-5' # 평일 오전 9시 KST (UTC 0시)
  workflow_dispatch:
permissions:
  contents: read
  issues: read
  pull-requests: read
jobs:
  report:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm install -g @anthropic-ai/claude-code
      - name: Generate report
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
          GH_TOKEN: ${ github.token }
          SLACK_WEBHOOK: ${ secrets.DAILY_REPORT_WEBHOOK }
        run: |
          chmod +x .github/scripts/daily-report.sh
          .github/scripts/daily-report.sh
```

```
# Cron 표현식 참고
# 0 0 * * 1-5 : 평일 (월~금) 오전 9시 KST
# 0 9 * * * : 매일 오후 6시 KST
# 0 1 * * 1 : 매주 월요일 오전 10시 KST

# 시간대 주의
# GitHub Actions의 cron은 UTC 기준
# KST = UTC + 9
# 한국 오전 9시 = UTC 0시

# 다양한 보고서 응용
# - 주간 보고서: cron '0 1 * * 1' (월요일)
# - 월간 보고서: cron '0 1 1 * *' (매월 1일)
# - 분기 보고서: 별도 트리거 또는 workflow_dispatch

# 메트릭 (1개월 운영 후 예시)
# - stand-up 시간: 평균 25분 → 12분
# - 정보 누락 빈도: 30% → 5%
# - 비활동자 식별: 자동
# - 매니저 보고 시간: 50% 절감
```

daily-report.yml

Cron schedule

정기

UTC

시간대

workflow_dispatch

수동

50% saved

시간 절감

패턴 조합과 운영

5가지 패턴을 함께 운영

COMBINATION

PR + 이슈

PR 머지 시 관련 이슈 자동 클로즈.
AI가 PR 본문에서 issue 번호 추출.

CASCADE

감사 → 이슈

보안 감사에서 발견된 취약점을 자동으로 GitHub Issue로 등록.

WORKFLOW

마이그레이션 → 보고서

마이그레이션 진행 상황을 매일 보고서에 포함해 가시성 확보.

METRIC

통합 대시보드

5가지 패턴 모두의 메트릭을 Grafana 등에 통합 표시.

GROWTH PATH

한 번에 5가지 모두 도입하지 마세요. PR 리뷰 봇으로 시작해 1개월 안정화 후 다음 패턴을 추가하는 점진 도입이 성공 비결입니다.

Combine
조합

Cascade
연쇄

Workflow
통합

Metric
측정

Part 5 Recap

자동화 패턴 핵심 정리

TAKEAWAY 1

5가지 완성 패턴

PR 리뷰, 이슈 트리아지, 마 이그레이션, 보안 감사, 매일 보고서

TAKEAWAY 2

구조화 JSON

모든 패턴이 AI 응답을 JSON으로 받아 후속 액션 자동화

TAKEAWAY 3

안전 장치

크기 제한, 자동 롤백, 테스트 검증, fork PR 방어

TAKEAWAY 4

점진 도입

한 패턴씩 안정화 후 확장. 메트릭으로 효과 검증

NEXT - PART 6

Part 6는 환경변수와 설정입니다. 모든 환경변수, 사내망 설정, OIDC 인증을 15장에 걸쳐 다룹니다.

Part 5

완료

Part 6

Env (15p)

Part 7-8

Debug + Labs (25p)

~1.5h

잔여

CHAPTER 05 - PART 06

CLI Reference – 환경변수와 설정

전체 CLI 플래그, Headless 모드, CI/CD 자동화



Part 6 / 환경변수와 설정

모든 환경변수와 사내망 통합

PART OBJECTIVE

Claude Code의 모든 환경변수를 체계적으로 정리합니다.

인증, 프록시, 사내 CA, 로그, 텔레메트리, AWS/GCP 통합 환경변수까지 모두 다룹니다.

TOPIC 1

인증 환경변수

API key, Bedrock 각 경로
의 환경변수

TOPIC 2

네트워크

HTTPS_PROXY,
NO_PROXY,
NODE_EXTRA_CA_CERTS

TOPIC 3

로그와 디버깅

LOG_LEVEL, telemetry
비활성화, 디버그 출력

TOPIC 4

커스터마이징

모델 별명, 도구 경로, 캐시
위치 같은 환경별 조정

15 Slides

Part 6 분량

~30 min

예상 강의 시간

20+ vars

환경변수

Reference

레퍼런스

인증 환경변수

3가지 경로별 설정

```
# 1. Anthropic Direct (기본)
$ export ANTHROPIC_API_KEY="sk-ant-api03-..."
$ claude -p "Hello"

# 2. AWS Bedrock 경로
$ export CLAUDE_CODE_USE_BEDROCK=1
$ export AWS_REGION=us-east-1
$ export AWS_ACCESS_KEY_ID=... # 또는 AWS Profile/Role
$ export AWS_SECRET_ACCESS_KEY=...
$ export AWS_SESSION_TOKEN=... # 임시 자격증명 (선택)
# 또는 AWS Profile
$ export AWS_PROFILE=production
$ claude -p "Hello"
# 사용 모델: anthropic.claude-3-5-sonnet-20241022-v2:0

# 3. GCP Vertex AI 경로
$ export CLAUDE_CODE_USE_VERTEX=1
$ export GOOGLE_APPLICATION_CREDENTIALS=/path/to/sa.json
$ export CLOUD_ML_REGION=us-central1
$ export ANTHROPIC_VERTEX_PROJECT_ID=my-project
$ claude -p "Hello"
```

```
# 4. 다중 경로 설정 (프로젝트별)
# 프로젝트 A: Bedrock
$ cd project-a && export CLAUDE_CODE_USE_BEDROCK=1
# 프로젝트 B: Direct
$ cd project-b && unset CLAUDE_CODE_USE_BEDROCK
$ export ANTHROPIC_API_KEY="sk-..."
```

```
# 5. direnv로 자동 전환 (.envrc)
$ cat .envrc
export CLAUDE_CODE_USE_BEDROCK=1
export AWS_REGION=us-east-1
export AWS_PROFILE=this-project
$ direnv allow
```

```
# 6. 우선순위
# 환경변수 > settings.json > 기본값
```

auth env

Direct

API key

Bedrock

AWS

Vertex

GCP

direnv

자동

AWS Bedrock 환경변수 심화

엔터프라이즈 설정

```
# 기본 Bedrock 설정
export CLAUDE_CODE_USE_BEDROCK=1
export AWS_REGION=us-east-1
```

```
# 1. 특정 모델 ID 지정
export ANTHROPIC_MODEL=us.anthropic.claude-sonnet-4-5-20250514-v1:0
# 또는 inference profile
export ANTHROPIC_MODEL=us.anthropic.claude-haiku-4-5-20251017-v1:0
```

```
# 2. Cross-region inference
# us. prefix : 미국 지역 자동 분산
# eu. prefix : 유럽 지역 자동 분산
# apac. prefix : 아시아-태평양 자동 분산
export ANTHROPIC_MODEL=apac.anthropic.claude-sonnet-4-5-20250514-v1:0
```

```
# 3. EC2 Instance Profile 사용 (IAM Role)
# 자격증명 명시 없이 자동 사용
# EC2/EKS/ECS에서 권장 (key 저장 안 함)
```

```
# 4. AWS SSO 통합
$ aws sso login --profile claude-prod
$ export AWS_PROFILE=claude-prod
$ claude doctor # AWS 자격증명 확인
```

```
# 5. STS AssumeRole (다른 계정 접근)
$ aws sts assume-role \
  --role-arn arn:aws:iam::ACCOUNT:role/ClaudeBedrockRole \
  --role-session-name claude-session > /tmp/creds
$ export AWS_ACCESS_KEY_ID=$(jq -r '.Credentials.AccessKeyId' /tmp/creds)
$ export AWS_SECRET_ACCESS_KEY=$(jq -r '.Credentials.SecretAccessKey' /tmp/creds)
$ export AWS_SESSION_TOKEN=$(jq -r '.Credentials.SessionToken' /tmp/creds)
```

Bedrock env

```
# 6. 엔드포인트 커스터마이징 (private link)
export BEDROCK_ENDPOINT_URL=https://bedrock-vpc.company.internal
```

```
# 7. 요청 timeout
export ANTHROPIC_BEDROCK_TIMEOUT_MS=60000
```

Cross-region

분산

IAM Role

권장

AssumeRole

계정 간

VPC endpoint

사내

네트워크 환경변수

사내망 통합 설정

```
# 1. HTTP/HTTPS Proxy
export HTTP_PROXY=http://proxy.company.com:8080
export HTTPS_PROXY=http://proxy.company.com:8080
```

```
# 인증이 필요한 프록시
export HTTPS_PROXY=http://user:pass@proxy.company.com:8080
# 또는 URL 인코딩
export HTTPS_PROXY=http://user:%40pass@proxy.company.com:8080
```

```
# 2. NO_PROXY (프록시 우회 도메인)
export NO_PROXY=localhost,127.0.0.1,.company.com,.internal
```

```
# 3. 사내 CA 인증서 (Self-signed 또는 사내 CA)
# 옵션 A: 단일 인증서
export NODE_EXTRA_CA_CERTS=/etc/ssl/certs/company-ca.pem
```

```
# 옵션 B: 여러 인증서 (PEM bundle)
export NODE_EXTRA_CA_CERTS=/etc/ssl/certs/ca-bundle.pem
```

```
# 4. TLS 검증 비활성화 (개발 환경만)
export NODE_TLS_REJECT_UNAUTHORIZED=0
# 프로덕션에서는 절대 금지
```

```
# 5. DNS 캐시
export NODE_DNS_CACHE_TIME=300 # 5분 캐시
```

```
# 6. 연결 timeout
export NODE_TIMEOUT_MS=30000 # 30초
```

```
# 7. IPv4 강제 (IPv6 문제 시)
export NODE_OPTIONS="--dns-result-order=ipv4first"
```

```
# 8. 디버깅 (자세한 네트워크 로그)
export DEBUG="claude:*,axios"
$ claude doctor 2>&1 | grep -i proxy
```

```
# 9. 트러블슈팅 체크리스트
# - curl -v https://api.anthropic.com 으로 직접 테스트
# - proxy 환경변수가 정확히 export됐는지 확인
# - 사내 CA가 제대로 로드되는지 doctor로 확인
# - NO_PROXY에 api 도메인이 잘못 포함되지 않았는지
```

network env

HTTPS_PROXY

프록시

NO_PROXY

우회

CA_CERTS

사내 CA

Timeout

대기

로그와 디버깅 환경변수

상세 로그 제어

log env

```
# 1. 로그 레벨
export CLAUDE_CODE_LOG_LEVEL=debug # debug|info|warn|error
# debug: 모든 상세 로그
# info: 일반 정보 (기본)
# warn: 경고 이상
# error: 오류만
```

```
# 2. 로그 파일 출력
export CLAUDE_CODE_LOG_FILE=/var/log/claude.log
# 미설정 시 stderr로만 출력
```

```
# 3. 텔레메트리 비활성화
export CLAUDE_CODE_DISABLE_TELEMETRY=1
# Anthropic으로 사용 통계 전송 안 함
```

```
# 4. 도구 호출 추적
export CLAUDE_CODE_TRACE_TOOLS=1
# 각 도구 호출 상세 로그
```

```
# 5. JSON 응답 추적
export CLAUDE_CODE_LOG_REQUESTS=1
# API 요청과 응답 전체를 로그
```

```
# 6. Node.js 디버깅
export NODE_OPTIONS="--inspect-brk"
# 디버거 attach 가능
```

```
# 7. 사용자 정의 user agent
export CLAUDE_CODE_USER_AGENT="company-internal/1.0"
# API 요청에 사용자 정의 헤더
```

```
# 8. 명령줄 옵션과의 관계
# --debug 옵션 = CLAUDE_CODE_LOG_LEVEL=debug
# --verbose 옵션 = CLAUDE_CODE_LOG_LEVEL=info + extra info
```

```
# 9. 로그 패턴 사용 예시
# 개발 (모든 로그)
export CLAUDE_CODE_LOG_LEVEL=debug
export CLAUDE_CODE_LOG_FILE=/tmp/claude-dev.log
```

```
# 프로덕션 (간결)
export CLAUDE_CODE_LOG_LEVEL=warn
export CLAUDE_CODE_LOG_FILE=/var/log/claude/prod.log
```

```
# CI (구조화 + 자세히)
export CLAUDE_CODE_LOG_LEVEL=info
export CLAUDE_CODE_LOG_REQUESTS=1
```

LOG_LEVEL

레벨

LOG_FILE

저장

TELEMETRY

통신

Per-env

환경별

모델 환경변수

기본 모델과 별명

```
# 1. 기본 모델 (settings.json도 가능)
export ANTHROPIC_MODEL=claude-sonnet-4-5
# 2. 모델별 사용 패턴
# 환경변수 우선순위
# --model 옵션 > ANTHROPIC_MODEL > settings.json model > default
# 3. settings.json의 modelAlias 활용
# ~/.claude/settings.json
{
  "modelAlias": {
    "fast": "claude-haiku-4-5",
    "smart": "claude-opus-4-5",
    "review": "claude-sonnet-4-5",
    "translate": "claude-haiku-4-5"
  }
}
# 사용 시
$ claude -p "..." --model fast
$ claude -p "..." --model smart
```

```
# 4. 모델 ID 표 (2026년 기준)
# Anthropic Direct
# - claude-sonnet-4-5 (기본)
# - claude-haiku-4-5 (저비용)
# - claude-opus-4-5 (최고 성능)

# AWS Bedrock
# - us.anthropic.claude-sonnet-4-5-20250514-v1:0
# - us.anthropic.claude-haiku-4-5-20251017-v1:0
# - us.anthropic.claude-opus-4-5-20250629-v1:0
# (cross-region inference profile 권장)

# 5. 모델 확인
$ claude doctor | grep -i model
$ claude -p "현재 사용 중인 모델은?" \
  --output-format json | jq -r '.model'
```

model env

ANTHROPIC_MODEL

기본

modelAlias

별명

Per-route

경로별 ID

doctor

확인

MCP 환경변수

서버별 자격증명 주입

1. MCP 서버 자격증명 (settings.json에서 참조)
~/.claude/settings.json

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "${GH_TOKEN}"
      }
    },
    "slack": {
      "env": {
        "SLACK_BOT_TOKEN": "${SLACK_BOT_TOKEN}"
      }
    },
    "postgres": {
      "env": {
        "POSTGRES_URL": "${DATABASE_URL}"
      }
    }
  }
}
```

2. 셸 환경변수로 주입

export GH_TOKEN=ghp...

export SLACK_BOT_TOKEN=xoxb-...

export DATABASE_URL=postgres://...

3. .env 파일 사용 (dotenv-cli)

.env

GH_TOKEN=ghp_xxx

SLACK_BOT_TOKEN=xoxb-yyy

명령 실행

\$ dotenv -e .env claude -p "..."

4. 사내 secret manager 통합

AWS Secrets Manager

\$ export GH_TOKEN=\$(aws secretsmanager get-secret-value \

--secret-id github-token --query SecretString --output text)

\$ claude -p "..."

HashiCorp Vault

\$ export GH_TOKEN=\$(vault kv get -field=token secret/github)

1Password CLI

\$ export GH_TOKEN=\$(op read "op://Engineering/GitHub/token")

5. CI/CD에서 (GitHub Actions)

Secrets로 등록 후

env:

GH_TOKEN: \${ secrets.GH_TOKEN }

SLACK_BOT_TOKEN: \${ secrets.SLACK_BOT_TOKEN }

6. 디버깅

\$ claude /mcp logs github # 토큰 인식 확인

MCP env

env ref

참조

.env

파일

Secrets Mgr

시크릿

CI secrets

CI

디렉토리와 캐시 환경변수

저장 위치 커스터마이징

dir env

```
# 1. 설정 디렉토리
export CLAUDE_CODE_HOME=~/.claude
# 기본값: ~/.claude
# 안에 들어가는 것: settings.json, glossary.md, sessions/
```

```
# 2. 세션 디렉토리
export CLAUDE_CODE_SESSIONS_DIR=~/.claude/sessions
# 세션별 메타데이터와 컨텍스트 저장
```

```
# 3. 캐시 디렉토리
export CLAUDE_CODE_CACHE_DIR=~/.cache/claude
# 모델 응답 캐시, MCP 결과 캐시
```

```
# 4. 로그 디렉토리
export CLAUDE_CODE_LOG_DIR=/var/log/claude
# 모든 로그 파일 저장 위치
```

```
# 5. 사내 표준 위치 (관리자 권한)
# /opt/claude/ : 시스템 전역 도구
# /etc/claude/ : 시스템 전역 설정 (managed-settings)
# /var/log/claude/ : 시스템 로그
# /var/cache/claude/ : 시스템 캐시
```

```
# 6. 캐시 정리
$ rm -rf $CLAUDE_CODE_CACHE_DIR
# 또는
$ claude cache clear
```

```
# 7. 캐시 비활성화 (디버깅 시)
export CLAUDE_CODE_DISABLE_CACHE=1
```

```
# 8. 임시 디렉토리
export TMPDIR=/var/tmp/claude
# claude가 사용하는 임시 파일 위치
# 기본값: /tmp 또는 시스템 기본
```

```
# 9. 디렉토리 권한 (보안)
$ chmod 700 ~/.claude # 본인만 접근
$ chmod 600 ~/.claude/settings.json # 본인만 읽기/쓰기
```

```
# 10. 디렉토리 백업
$ tar czf claude-backup.tar.gz ~/.claude
# settings, glossary 등 백업
```

CLAUDE_CODE_HOME

설정

CACHE_DIR

캐시

LOG_DIR

로그

chmod 700

권한

환경변수 점검 명령

claude doctor 활용

```
# 1. 종합 진단
$ claude doctor
Claude Code v1.2.0
✓ Node.js v20.10.0
✓ Settings: ~/.claude/settings.json (valid)
✓ Authentication:
  - Type: AWS Bedrock
  - Region: us-east-1
  - Profile: production
✓ Network:
  - HTTPS_PROXY: http://proxy.company.com:8080
  - NO_PROXY: localhost,.company.com
  - CA: /etc/ssl/certs/company-ca.pem
✓ MCP servers: 5 connected
  - github: ✓
  - slack: ✓
  - jira: ✓ (auth via env)
  - postgres: ✓
  - filesystem: ✓
△ Log level: debug (운영에서는 info 권장)
× Disk space: 92% used
```

```
# 2. 상세 모드
$ claude doctor --verbose
# Settings 전체 출력 + 환경변수 값 + 모든 MCP 도구 목록
# 3. 환경변수만 확인
$ env | grep -i claude
$ env | grep -i anthropic
$ env | grep -i aws
$ env | grep -i proxy
# 4. 한 번에 모두 출력
$ env | grep -iE 'claude|anthropic|aws|gcp|proxy|ca_cert'

# 5. settings 파일 위치 확인
$ claude doctor --verbose | grep -i settings
$ ls -la ~/.claude/

# 6. 디버깅 모드로 환경변수 로딩 확인
$ CLAUDE_CODE_LOG_LEVEL=debug claude --version 2>&1 | \
  grep -i 'loading'
# 7. 트러블슈팅 체크리스트
- export가 셸에 적용됐는지 (.bashrc, .zshrc 확인)
- 새 터미널에서도 환경변수가 보이는지
- direnv 사용 시 direnv allow 했는지
- /etc/environment의 시스템 전역 변수
- CI에서는 secrets로 등록했는지
```

inspect

doctor

종합

--verbose

상세

env grep

필터

Checklist

점검

환경별 설정 패턴

dev/staging/prod 분리

```
# 1. shell 스크립트로 환경별 로드
# ~/.bin/claude-env.sh
case "$1" in
  dev)
    export
    ANTHROPIC_API_KEY="$DEV_API_KEY"
    export CLAUDE_CODE_LOG_LEVEL=debug
    export ANTHROPIC_MODEL=claude-haiku-4-5
    ;;
  staging)
    export CLAUDE_CODE_USE_BEDROCK=1
    export AWS_PROFILE=claude-staging
    export CLAUDE_CODE_LOG_LEVEL=info
    ;;
  prod)
    export CLAUDE_CODE_USE_BEDROCK=1
    export AWS_PROFILE=claude-prod
    export CLAUDE_CODE_LOG_LEVEL=warn
    export ANTHROPIC_MODEL=claude-sonnet-4-5
    ;;
esac
```

```
# 사용
$ source ~/.bin/claude-env.sh prod
$ claude doctor

# 2. direnv로 디렉토리별 자동 적용
# 프로젝트 폴더의 .envrc
case "$PWD" in
  */project-a/*)
    use_aws_profile claude-prod
    export ANTHROPIC_MODEL=claude-opus-4-5
    ;;
  */project-b/*)
    export ANTHROPIC_API_KEY="$DEV_API_KEY"
    ;;
esac
```

```
# 3. 사내 표준 wrapper
# /opt/company/bin/claude
#!/bin/bash
# 사내 표준 환경변수 강제 적용
export CLAUDE_CODE_USE_BEDROCK=1
export AWS_REGION=us-east-1
export NODE_EXTRA_CA_CERTS=/etc/ssl/company-ca.pem
export HTTPS_PROXY=http://proxy.company.com:8080
export NO_PROXY=localhost,.company.com
export CLAUDE_CODE_DISABLE_TELEMETRY=1

# 사용자가 임의 변경 못 하게
exec /opt/claude/node_modules/.bin/claude "$@"

# 4. Docker 컨테이너에서
# Dockerfile
FROM node:20-alpine
ENV CLAUDE_CODE_USE_BEDROCK=1
ENV AWS_REGION=us-east-1
ENV NO_PROXY=localhost
RUN npm install -g @anthropic-ai/claude-code
ENTRYPOINT ["claude"]
```

env patterns

Shell script

수동

direnv

자동

Wrapper

강제

Docker

격리

환경변수 보안

안전한 시크릿 관리

RISK 1

히스토리 노출

export
ANTHROPIC_API_KEY=... 은
~/.bash_history에 기록됨.
read -s로 입력

RISK 2

env 명령 노출

ps aux나 env에서 다른 사용자
가 볼 수 있음.
/proc/PID/environ 권한 확인

RISK 3

코드 커밋

.envrc, .env를 .gitignore
예. gitleaks pre-commit
hook으로 차단

RISK 4

로그 노출

CLAUDE_CODE_LOG_RE
QUESTS=1 시 응답에 시크
릿 포함 가능.
운영에서 비활성화

```
# 안전한 입력 (히스토리 회피)
$ read -s -p "API key: " ANTHROPIC_API_KEY && export ANTHROPIC_API_KEY
# pass(password-store) 통합
$ export ANTHROPIC_API_KEY=$(pass show anthropic/api-key)
# macOS Keychain
$ export ANTHROPIC_API_KEY=$(security find-generic-password -w -s anthropic)
```

safe input

History

히스토리

Process

프로세스

Commit

커밋

Log

로그

환경변수 트러블슈팅

자주 발생하는 문제

```
# 문제 1: 환경변수가 안 보임
# 증상: claude doctor에서 ANTHROPIC_API_KEY가 not set
# 원인 1: export 안 함
$ ANTHROPIC_API_KEY=sk-... # 잘못
$ export ANTHROPIC_API_KEY=sk-... # 올바른
# 원인 2: 새 셸에서 적용 안 됨
$ source ~/.bashrc
# 또는 ~/.bashrc에 export 추가 후 새 터미널
# 원인 3: sudo 후 환경변수 안 가져감
$ sudo -E claude doctor # -E 옵션으로 환경 유지

# 문제 2: 프록시 통과 안 됨
# 증상: connect ETIMEDOUT
# 진단
$ curl -v https://api.anthropic.com 2>&1 | head -20
# 원인 확인
$ echo $HTTPS_PROXY
$ echo $NO_PROXY
$ env | grep -i proxy
# 해결
$ export HTTPS_PROXY=http://proxy.company.com:8080
$ export NO_PROXY=localhost,127.0.0.1,.internal
```

```
# 문제 3: 사내 CA 인식 안 됨
# 증상: unable to verify the first certificate
```

troubleshooting

```
$ openssl x509 -in /etc/ssl/company-ca.pem -text -noout
# 인증서가 유효한지 확인
```

```
$ export NODE_EXTRA_CA_CERTS=/etc/ssl/company-ca.pem
$ claude doctor
```

```
# 문제 4: Bedrock 자격증명 만료
# 증상: ExpiredTokenException
```

```
$ aws sso login --profile production
$ export AWS_PROFILE=production
```

```
# 문제 5: direnv 적용 안 됨
$ direnv allow .
$ direnv reload
```

```
# 종합 디버그
$ CLAUDE_CODE_LOG_LEVEL=debug claude doctor 2>&1 | less
```

No export

흔한 실수

sudo -E

권한 유지

curl -v

직접 검증

debug

추적

환경변수 베스트 프랙티스

운영 권장 사항

BP 1

명시적 export

shell 시작 스크립트(.bashrc, .zshrc)에 export.
session마다 다시 입력 불필요

BP 2

도구로 관리

direnv, pass, 1Password
CLI 같은 도구 활용.
평문 저장 회피

BP 3

환경별 분리

dev, staging, prod에 다른 시크릿.
사고 시 영향 범위 제한

BP 4

정기 회전

API key, 토큰을 90일마다 회전.
자동화로 잊지 않게

NEVER

API key를 코드에 하드코딩하거나 Slack, 이메일로 공유하지 마세요. PR diff에 노출되면 즉시 회전하고 보안팀에 보고합니다.

Explicit

명시

Tools

도구

Separate

분리

Rotate

회전

환경변수 종합 표

모든 환경변수 한눈에

환경변수	용도	기본값
ANTHROPIC_API_KEY	Direct API 인증	필수
ANTHROPIC_MODEL	기본 모델	sonnet
CLAUDE_CODE_USE_BEDROCK	AWS Bedrock 경로	0
CLAUDE_CODE_USE_VERTEX	GCP Vertex 경로	0
AWS_PROFILE / AWS_REGION	AWS 자격증명	필수 (Bedrock)
HTTPS_PROXY / NO_PROXY	프록시	미설정
NODE_EXTRA_CA_CERTS	사내 CA	미설정
CLAUDE_CODE_LOG_LEVEL	로그 레벨	info
CLAUDE_CODE_DISABLE_TELEMETRY	텔레메트리 차단	0

9 vars

핵심

doctor

확인

Per-env

환경별

Reference

레퍼런스

Part 6 Recap

환경변수와 설정 핵심 정리

TAKEAWAY 1

인증 환경변수

Direct, Bedrock와 각각의
변수

TAKEAWAY 2

네트워크

HTTPS_PROXY, NO_PROXY,
NODE_EXTRA_CA_CERTS로
사내망 통합

TAKEAWAY 3

환경별 패턴

shell 스크립트, direnv,
wrapper, Docker로 환경
분리

TAKEAWAY 4

보안

export 도구, 정기 회전, 평문
저장 회피, 환경별 분리

NEXT - PART 7

Part 7은 디버깅과 트러블슈팅입니다. 일반 문제 6가지의 진단과 해결법, --debug 활용을 10장에 걸쳐 다룹니다.

Part 6

완료

Part 7

Debug (10p)

Part 8

Recap + Labs (15p)

~1h

잔여

CHAPTER 05 - PART 07

CLI Reference – 디버깅/트러블슈팅

전체 CLI 플래그, Headless 모드, CI/CD 자동화



Part 7 / 디버깅과 트러블슈팅

문제 발생 시 진단 흐름

PART OBJECTIVE

CLI 사용 중 발생하는 일반 문제 6가지의 진단과 해결법을 학습합니다. --debug, doctor, 로그 분석, 에스컬레이션 흐름을 모두 다룹니다.

TOPIC 1

진단 도구

doctor, --debug, --
verbose, 로그 파일 분석

TOPIC 2

인증 문제

401, 403, 만료 토큰, 잘못
된 region

TOPIC 3

네트워크 문제

프록시, CA, DNS,
timeout, IPv6

TOPIC 4

동작 문제

응답 없음, 잘못된 결과, 도
구 실패, 한도 도달

10 Slides

Part 7 분량

~20 min

예상 강의 시간

6 problems

일반 문제

Diagnostic

진단

진단 흐름

문제 발생 시 5단계

STEP 01

claude
doctor

STEP 02

--debug
모드

STEP 03

로그
분석

STEP 04

직접
검증

STEP 05

에스컬레이션

```
# 1단계: 종합 진단
$ claude doctor
# 모든 영역(인증, 네트워크, MCP)을 한 번에 체크

# 2단계: 상세 디버그 모드
$ claude --debug -p "..." 2> /tmp/claude-debug.log
# stderr에 모든 내부 동작 기록

# 3단계: 로그 분석
$ grep -i error /tmp/claude-debug.log
$ tail -100 /tmp/claude-debug.log
```

```
# 4단계: 직접 검증
$ curl -v https://api.anthropic.com/v1/messages \
  -H "x-api-key: $ANTHROPIC_API_KEY" \
  -H "anthropic-version: 2023-06-01" \
  -d '{"model": "...", "max_tokens": 10,
    "messages": [{"role": "user", "content": "hi"}]}'

# 5단계: 에스컬레이션
# - 사내 Slack #claude-help
# - Anthropic Support (https://support.anthropic.com)
# - GitHub Issues (anthropics/claude-code)
```

diagnostic flow

doctor

1단계

debug

2단계

log

3단계

curl

4단계

인증 문제 진단

401/403 에러

auth issues

```
# Symptom: "Authentication failed" or 401 Unauthorized
# 진단 1: API key 확인
$ echo "${ANTHROPIC_API_KEY:0:15}..." # 일부만 표시
# 잘못된 예: 빈 값, 형식 오류, 따옴표 포함
```

```
# 진단 2: API key 직접 테스트
$ curl -sS https://api.anthropic.com/v1/messages \
  -H "x-api-key: $ANTHROPIC_API_KEY" \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-haiku-4-5", "max_tokens": 10, "messages": [{"role": "user", "content": "hi"}]}' \
  | jq .
```

```
# 401 응답: API key가 유효하지 않음
# 200 응답: API key는 유효, 다른 문제
```

```
$ aws bedrock list-foundation-models --region us-east-1 \
  | jq '.modelSummaries[] | select(.modelId | contains("claude"))'
# Claude 모델이 활성화되었는지 확인
```

```
# Bedrock 401/403 시
$ aws sts get-caller-identity
# 응답이 있으면 자격증명 OK
# ExpiredToken이면 SSO 재로그인
```

```
# 진단 3: doctor로 종합 확인
$ claude doctor --verbose | grep -i auth
```

```
# 해결 패턴
1. API key 새로 발급 (Anthropic Console)
2. 환경변수 재export
3. Bedrock: aws sso login
4. settings.json의 model이 region과 일치하는지
```

echo
key 확인

curl
직접 검증

aws sts
AWS

gcloud auth
GCP

네트워크 문제 진단

Timeout, CA, 프록시

Symptom: connect ETIMEDOUT, unable to verify certificate

진단 1: 기본 네트워크 도달성

\$ curl -v https://api.anthropic.com 2>&1 | head -20

빠른 응답: 네트워크 OK

Timeout: 프록시 문제 가능성

진단 2: 프록시 환경변수

\$ env | grep -i proxy

HTTPS_PROXY가 설정됐는지, 형식 정확한지 확인

진단 3: 사내 CA 인식

\$ openssl s_client -connect api.anthropic.com:443 \
-CAfile \$NODE_EXTRA_CA_CERTS < /dev/null

Verify return code: 0 (ok) 면 CA 정상

진단 4: DNS 확인

\$ nslookup api.anthropic.com

\$ dig api.anthropic.com

진단 5: IPv6 문제 (드물지만 발생)

\$ curl -4 https://api.anthropic.com # IPv4 강제

\$ curl -6 https://api.anthropic.com # IPv6 강제

해결 1: 프록시 설정

\$ export HTTPS_PROXY=http://proxy.company.com:8080

\$ export NO_PROXY=localhost,127.0.0.1,.internal

해결 2: 사내 CA 추가

\$ export NODE_EXTRA_CA_CERTS=/etc/ssl/company-ca.pem

해결 3: IPv4 강제

\$ export NODE_OPTIONS="--dns-result-order=ipv4first"

해결 4: timeout 늘림

\$ export NODE_TIMEOUT_MS=60000

종합 디버깅

\$ CLAUDE_CODE_LOG_LEVEL=debug claude doctor 2>&1 \
| grep -iE 'proxy|cert|timeout|dns'

network issues

curl -v

기본

openssl

CA 검증

nslookup

DNS

IPv4 force

드문 케이스

응답 동작 문제

응답 없음, 잘못된 결과

```
# Symptom 1: 응답이 비어 있음 (.result is empty)
# 원인과 해결
$ claude -p "... " --output-format json | jq -r '.result'
# 빈 응답이면:

$ claude -p "... " --output-format json | jq '.subtype, .is_error'
# subtype이 error: 도구 실패나 권한 거부
# is_error: true: 시스템 에러

$ claude -p "... " --output-format json | jq '.num_turns'
# 0이면 모델이 응답하지 않음 (입력 문제 가능)

# Symptom 2: 결과가 이상함 (할루시네이션)
# 진단: 입력 컨텍스트 확인
$ claude --debug -p "... " 2>&1 | grep -i context
# 모델이 본 컨텍스트가 정확한가?

# 해결: 명시적 가이드 추가
$ claude -p "다음 데이터만 사용해 응답. 추측 금지:
$(cat data.json)
질문: ..."
```

Symptom 3: 도구 호출 실패

```
$ claude --debug -p "... " 2>&1 | grep -i tool_use
$ claude --debug -p "... " 2>&1 | grep -i tool_result
```

결과 확인

```
{"type":"tool_result", "is_error": true, "content": "..."}
# error 내용에 단서
```

해결: 권한이나 매개변수 확인

```
$ claude -p "... " --allowed-tools "Read,Grep" --debug
```

Symptom 4: 한도 도달 (max_turns_reached)

```
$ claude -p "... " --max-turns 50 # 더 많이 허용
# 또는 작업을 분할
$ S=$(claude -p "1단계" --output-format json | jq -r '.session_id')
$ claude --continue "$S" -p "2단계"
```

response issues

subtype

응답 분류

is_error

에러

num_turns

호출

Split

분할

트러블슈팅 종합 가이드

문제별 빠른 참조

증상	원인 후보	해결
Authentication failed	API key, 토큰 만료, 잘못된 region	sso login, key 재발급
connect ETIMEDOUT	프록시, 방화벽, DNS	HTTPS_PROXY 설정
unable to verify cert	사내 CA 미설치	NODE_EXTRA_CA_CERTS
Tool not allowed	permissions deny	--allowed-tools 또는 settings
max_turns_reached	복잡한 작업	--max-turns 늘림 or 분할
MCP server failed	환경변수, 네트워크	/mcp logs로 확인
응답 너무 느림	큰 컨텍스트, 느린 도구	입력 줄이기, Haiku 사용
비용 폭증	Opus 사용, 큰 입력	모델 변경, max-tokens

8 cases

주요 증상

Quick ref

빠른 참조

Causes

원인

Solutions

해결

디버그 정보 수집

에스컬레이션 시 첨부할 내용

```
#!/bin/bash
# collect-debug-info.sh - 종합 디버그 정보 수집
# 사내 Slack이나 Support 에스컬레이션 시 사용

REPORT=/tmp/claude-debug-$(date +%Y%m%d-%H%M%S).txt
```

```
{
  echo "=== Claude Code Debug Info ==="
  echo "Date: $(date)"
  echo "User: $(whoami)"
  echo "Host: $(hostname)"
  echo ""

  echo "=== Versions ==="
  echo "Node: $(node --version)"
  echo "Claude: $(claude --version)"
  echo "OS: $(uname -a)"
  echo ""

  echo "=== Environment (filtered) ==="
  env | grep -iE 'claude|anthropic|aws_region|aws_profile|proxy|node_extra' \
    | sed 's/=.\\{6,\\.\\}.*$/=<redacted>/'
  echo ""
}
```

```
echo "=== Settings ==="
cat ~/.claude/settings.json 2>/dev/null | \
  jq 'del(.. | objects | select(has("token") or has("key"))) | .)'
echo ""
echo "=== Doctor ==="
claude doctor --verbose 2>&1
echo ""
echo "=== Recent Log (last 100 lines) ==="
tail -100 ~/.claude/logs/$(date +%Y-%m-%d).log 2>/dev/null
echo ""
echo "=== Network Test ==="
curl -sSI https://api.anthropic.com 2>&1
echo ""
echo "=== Failed command (manual paste) ==="
echo "<여기에 실패한 명령과 결과 붙여넣기>"
} > "$REPORT"

echo "✓ Debug report: $REPORT"
echo "업로드 방법: 사내 Slack #claude-help"

# 시크릿 마스크 확인
$ grep -i 'sk-\\|akia\\|password\\|token=' "$REPORT"
# 비어 있으면 안전
```

collect-debug

Comprehensive

종합

Redacted

마스킹

Single file

한 파일

Sharable

공유 안전

자주 묻는 질문

CLI 관련 FAQ

FAQ

Q1. -p와 --print의 차이는?

A. 동일합니다. -p는 단축형, 둘 다 Headless 모드 활성화.

Q2. 응답을 파일로 저장하면 한국어가 깨져요.

A. locale 문제. export LANG=ko_KR.UTF-8 또는 LC_ALL=en_US.UTF-8 후 재시도.

Q3. JSON 응답에서 result가 잘려요.

A. --max-tokens 기본 4096 한계. --max-tokens 8000 등으로 늘림.
또는 응답이 진짜 그 길이로 끝났는지 stop_reason 확인.

Q4. 같은 프롬프트인데 매번 다른 결과가 나옵니다.

A. Claude는 stochastic. 동일 응답이 필요하면 --temperature 0.
실제로는 거의 같지만 약간 다를 수 있음.

Q5. 도구 호출 결과가 컨텍스트에 안 보여요.

A. --debug로 tool_result 이벤트 확인.
너무 큰 결과는 자동 trimming. 작은 부분만 요청.

Q6. CI에서 claude가 매번 새로 설치되어 느립니다.

A. GitHub Actions cache 또는 사내 도커 이미지 베이킹.
한 번 빌드 후 재사용.

Q7. Windows에서 동작하나요?

A. Windows 10/11에서 WSL2 권장. PowerShell도 가능하지만
pipe 사용 시 인코딩 차이로 종종 문제.

Q8. 비용을 미리 알 수 있나요?

A. --output-format json의 usage 필드에 토큰 수 포함.
모델별 단가와 곱해 호출별 비용 계산. 또는 Anthropic Console.

8 Qs

대표

Quick

즉답

Practical

실용

Reference

참고

도움 받기

에스컬레이션 경로

LEVEL 1

자가 진단

doctor, --debug, 로그 분석, FAQ 확인으로 대부분 해결

LEVEL 2

사내 채널

#claude-help Slack 채널, 사내 위키, 동료 개발자

LEVEL 3

커뮤니티

GitHub Discussions, Stack Overflow, Anthropic Discord

LEVEL 4

공식 지원

support.anthropic.com 또는 사내 계약 채널

WHEN TO ESCALATE

자가 진단을 시도하고 30분 이상 막혀 있거나, 같은 문제가 반복되거나, 보안/비용 영향이 큰 경우 즉시 에스컬레이션하세요.

Self

1단계

Team

2단계

Community

3단계

Vendor

4단계

Part 7 Recap

디버깅과 트러블슈팅 핵심 정리

TAKEAWAY 1

5단계 진단

doctor → --debug → 로
그 분석 → 직접 검증 → 에
스컬레이션

TAKEAWAY 2

주요 문제 6가지

인증, 네트워크, CA, 도구,
한도, 응답 동작 진단법

TAKEAWAY 3

디버그 정보

collect-debug-info.sh로
안전한 종합 정보 수집

TAKEAWAY 4

도움 받기

자가 → 사내 → 커뮤니티 →
공식 지원 단계별 활용

NEXT - PART 8

Part 8은 Chapter 5 핵심 요약과 4가지 Hands-on Lab입니다. 실제로 손에 익히는 마지막 시간입니다.

Part 7

완료

Part 8

Recap + Labs

4 Labs

실습

~30 min

잔여

Chapter 5 핵심 요약

요약

BASICS

CLI 기본

4가지 호출 방식, 6대 플래그 카테고리, JSON 파싱 마스터

AUTOMATION

자동화

Headless 5가지 스크립트, 5가지 실전 패턴, CI/CD 3 플랫폼

PARSING

출력 파싱

JSON 스키마, stream-json, jq 고급, CSV/HTML 변환

OPS

운영

환경변수 종합, 디버깅 5단계, 보안과 비용 통제

OUTCOME

여러분은 이제 Claude Code를 명령줄 자동화 도구로 활용할 수 있습니다. PR 리뷰, 보안 감사, 매일 보고서를 모두 자동화하고 CI/CD에 통합할 수 있습니다.

8 Parts

완료

130 Slides

Chapter 5

~3.5h

강의 시간

4 Labs

실습 시작

CHAPTER 05

CLI Reference – Summary & LAB

전체 CLI 플래그, Headless 모드, CI/CD 자동화



Hands-on Labs / 4종

직접 손에 익히는 시간

Lab	내용	예상 시간
Lab 1 / 첫 Headless 호출	-p 옵션과 JSON 파싱, 기본 패턴 익히기	20분
Lab 2 / 자동화 스크립트	PR 리뷰 스크립트 작성과 실행	30분
Lab 3 / CI 통합	GitHub Actions 워크플로 작성	30분
Lab 4 / 데이터 파싱	JSON → CSV → 대시보드 생성	20분

TOTAL

4개 랩 합산 약 100분. 본인 GitHub 저장소에서 진행하면 실무 적용도가 높습니다. 어려움이 있을 때 즉시 멘토에게 도움 요청.

4 Labs

실습 수

~100 min

총 시간

Hands-on

실습 중심

Mentor

도움

Lab 1 / 첫 Headless 호출

기본 패턴 익히기 (20분)

Lab 1

```
# 1. 단순 호출
$ claude -p "Python에서 리스트와 튜플의 차이를 3줄로"
# 결과: 마크다운 텍스트

# 2. JSON 출력
$ claude -p "Python 리스트와 튜플 비교" \
  --output-format json | jq .

# 3. result만 추출
$ claude -p "..." --output-format json | jq -r '.result'

# 4. 토큰 사용량 확인
$ claude -p "..." --output-format json | \
  jq '.usage | "in=\(.input_tokens) out=\(.output_tokens)'"

# 5. 파일로 저장
$ claude -p "README 초안 작성" \
  --output-format json | jq -r '.result' > README.md

# 6. pipe 입력
$ cat package.json | claude -p "이 의존성 보안 검토"
```

```
# 7. 환경변수 활용
$ MODEL=claude-haiku-4-5 claude -p "..." --model "$MODEL"

# 8. 단단계 (세션 재사용)
$ S=$(claude -p "Python 정렬 알고리즘 5가지" \
  --output-format json | jq -r '.session_id')
$ claude --continue "$S" -p "그 중 가장 빠른 2개의 코드"

# 9. doctor로 환경 확인
$ claude doctor

# 10. 도움말 탐색
$ claude --help
$ claude doctor --help
```

20 min

예상 소요

10 patterns

연습

Hands-on

직접 실행

Foundation

기초

Lab 2 / 자동화 스크립트

PR 리뷰 스크립트 (30분)

```
#!/bin/bash
# my-pr-review.sh - 본인이 직접 작성하는 PR 리뷰
set -euo pipefail

PR_NUM="${1:?Usage: my-pr-review.sh <PR-number>}"

# Step 1: PR 정보 가져오기
echo "PR #${PR_NUM} 정보 수집..."
PR_INFO=$(gh pr view "${PR_NUM}" --json title,additions,deletions)
TITLE=$(echo "$PR_INFO" | jq -r '.title')
echo "제목: $TITLE"

# Step 2: 크기 체크
TOTAL=$(echo "$PR_INFO" | jq '.additions + .deletions')
if [ "$TOTAL" -gt 1000 ]; then
    echo "⚠️ PR이 큼니다 ($TOTAL 줄)"
    read -p "계속할까요? (y/N) " ANS
    [ "$ANS" != "y" ] && exit 0
fi
```

```
# Step 3: Claude로 리뷰
echo "Claude 리뷰 시작..."
REVIEW=$(claude -p "$(cat <<EOF
PR #${PR_NUM} "$TITLE" 리뷰.
```

```
Diff:
$(gh pr diff "${PR_NUM}")
```

```
관점: 1) 코드 품질 2) 보안 3) 테스트 4) 문서
출력: Markdown, 우선순위에별 정리
EOF
)" \
```

```
--allowed-tools "Read,Grep,Glob" \
--max-turns 10 \
--output-format json | jq -r '.result')
```

```
# Step 4: 결과 출력
echo ""
echo "=== Review ==="
echo "$REVIEW"
```

Lab 2

30 min

예상

5 steps

단계

gh CLI

GitHub

Iterative

확인

Lab 2 / 자동화 스크립트

PR 리뷰 스크립트 (30분)

Lab 2

```
# Step 4: 결과 출력
echo ""
echo "=== Review ==="
echo "$REVIEW"

# Step 5: 코멘트 게시 여부 확인
read -p "PR에 코멘트 게시할까요? (y/N) " POST
if [ "$POST" = "y" ]; then
    gh pr comment "$PR_NUM" --body "$REVIEW"
    echo "✓ 코멘트 게시 완료"
fi

# 활용 예시
# $ chmod +x my-pr-review.sh
# $ ./my-pr-review.sh 1234
```

30 min

예상

5 steps

단계

gh CLI

GitHub

Iterative

확인

Lab 3 / CI 통합

GitHub Actions 워크플로 (30분)

```
# 1. 저장소에 secret 등록
# GitHub → Settings → Secrets and variables → Actions
# Name: ANTHROPIC_API_KEY
# Value: sk-ant-api03-...
```

```
# 2. .github/workflows/my-claude.yml 작성
name: My Claude Workflow
```

```
on:
  pull_request:
    types: [opened]
  workflow_dispatch: # 수동 실행도 가능
```

```
permissions:
  pull-requests: write
  contents: read
```

```
jobs:
  test-claude:
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - name: Install Claude
        run: npm install -g @anthropic-ai/claude-code
      - name: Test Claude
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          # 간단한 테스트
          echo "안녕하세요" | claude -p "한국어 인사를 영어로" \
            --output-format json | jq -r '.result'
      - name: Show usage
        if: always()
        run: |
          echo "Workflow 완료 시각: $(date)"
```

Lab 3

30 min

예상

Secret

인증

Workflow

작성

PR test

검증

```
# 3. PR 만들어서 자동 실행 확인
$ git checkout -b test-claude-action
$ git add .github/workflows/my-claude.yml
$ git commit -m "Add test workflow"
$ git push origin test-claude-action
$ gh pr create --title "Test Claude action" --body "Testing"
```

Lab 3 / CI 통합

GitHub Actions 워크플로 (30분)

```
# 3. PR 만들어서 자동 실행 확인
$ git checkout -b test-claude-action
$ git add .github/workflows/my-claude.yml
$ git commit -m "Add test workflow"
$ git push origin test-claude-action
$ gh pr create --title "Test Claude action" --body "Testing"
```

```
# 4. Actions 탭에서 결과 확인
$ gh run list
$ gh run watch # 실시간 확인
```

Lab 3

30 min

예상

Secret

인증

Workflow

작성

PR test

검증

Lab 4 / 데이터 파싱

JSON → CSV → 대시보드 (20분)

Lab 4

```
#!/bin/bash
# generate-dashboard.sh - 응답 데이터 시각화

mkdir -p data

# Step 1: 여러 호출 실행 (데이터 수집)
for topic in "Python" "JavaScript" "Go" "Rust" "TypeScript"; do
  claude -p "$topic 언어의 장점 3가지" \
    --output-format json > "data/$topic.json"
done

# Step 2: JSON → CSV 변환
{
  echo "language,duration_ms,input_tokens,output_tokens,model"
  jq -rs '.[[]] | [
    input_filename | sub("data/"; "") | sub(".json"; ""),
    .duration_ms,
    .usage.input_tokens,
    .usage.output_tokens,
    .model
  ] | @csv' data/*.json
} > stats.csv

cat stats.csv
```

```
# Step 3: 통계 계산
echo ""
echo "=== 통계 ==="
jq -rs '
{
  total_calls: length,
  avg_duration_ms: ([.[] .duration_ms] | add / length | floor),
  total_input: ([.[] .usage.input_tokens] | add),
  total_output: ([.[] .usage.output_tokens] | add)
}
' data/*.json | jq .
```

20 min

예상

JSON→CSV

변환

Stats

통계

HTML

시각화

Lab 4 / 데이터 파싱

JSON → CSV → 대시보드 (20분)

Lab 4

```
# Step 4: HTML 대시보드 생성
DATA=$(jq -rs '...' data/*.json)
cat <<HTML > dashboard.html
<!DOCTYPE html>
<html><head>
<title>Claude Usage Dashboard</title>
<style>
body { font-family: sans-serif; max-width: 800px; margin: auto; }
table { border-collapse: collapse; width: 100%; }
th, td { padding: 8px; border-bottom: 1px solid #ddd; }
th { background: #f0f0f0; }
</style>
</head><body>
<h1>📊 Claude API 사용 통계</h1>
<p>생성: $(date)</p>
$(cat stats.csv | awk -F, 'BEGIN{print "<table>"
  NR==1{print "<tr><th>"$1"</th><th>"$2"</th></tr>"; next}
  {print "<tr><td>"$1"</td><td>"$2"</td></tr>"}
  END{print "</table>"}')
</body></html>
HTML
echo "✓ dashboard.html 생성 완료"
echo " 열기: open dashboard.html (macOS) / start dashboard.html (Windows)"
```

20 min

예상

JSON→CSV

변환

Stats

통계

HTML

시각화

실무 적용 로드맵

4주 점진 도입

STEP 01

오늘
환경 설정

STEP 02

1주차
수동 활용

STEP 03

2주차
스크립트화

STEP 04

3주차
CI 통합

STEP 05

4주차
팀 공유

WEEK 1

개인 활용

Headless 모드로 일상 작업
자동화. 자주 쓰는 명령 패턴
정착

WEEK 2

스크립트화

반복 작업 5가지를 스크립
트로. PR 리뷰, 로그 분석 등

WEEK 3

CI 통합

저장소 1개에 GitHub
Actions 통합. 자동 PR 리
뷰 등

WEEK 4

팀 공유

스크립트와 워크플로를 팀
과 공유. 도입 효과 측정

Day 1

시작

Week 1

개인

Week 2-3

자동화

Week 4

팀

비용 통제 마지막 점검

실수 회피 체크리스트

CHECK 1

모델 선택

단순 작업은 Haiku, 복잡한
분석만 Sonnet, 핵심만
Opus

CHECK 2

컨텍스트 최소화

grep, awk, jq로 사전 필터
링. 전체 파일 입력 회피

CHECK 3

한도 설정

--max-turns 10-20, --
max-tokens 4000-8000
명시

CHECK 4

모니터링

usage 토큰 로깅, 일일/월
간 한도 알람, 비용 추적

```
# 비용 계산 헬퍼 함수 (.bashrc에 추가)
claude_cost() {
  jq -r '.usage as $u | .model as $m |
    if ($m | startswith("claude-haiku")) then
      ($u.input_tokens * 1 + $u.output_tokens * 5) / 1e6
    elif ($m | startswith("claude-sonnet")) then
      ($u.input_tokens * 3 + $u.output_tokens * 15) / 1e6
    elif ($m | startswith("claude-opus")) then
      ($u.input_tokens * 15 + $u.output_tokens * 75) / 1e6
    else 0 end'
}
# 사용: claude -p "..." --output-format json | claude_cost
# 결과: 0.0023 (약 $0.0023)
```

cost helper

Model
선택

Context
최소화

Limits
한도

Monitor
추적

보안 마지막 점검

8가지 핵심

No	항목	확인
1	API key는 환경변수로만	코드 평문 금지
2	CI는 secrets 사용	YAML에 직접 X
3	환경별 분리 (dev/stage/prod)	다른 key
4	--allowed-tools로 권한 좁힘	Read만 안전
5	--dangerously-skip-permissions 금지	CI 사고 흔함
6	fork PR 시 secrets 차단 확인	GitHub 기본
7	로그에서 시크릿 마스킹	collect-debug 활용
8	정기 회전 (90일)	자동화로 강제

8 items
체크리스트

Layered
계층 방어

Mandatory
필수

Audit
감사

운영 메트릭

도입 효과를 측정

METRIC 1

시간 절감

자동화로 절약된 시간 =
(수동 작업 시간 - AI 시간)
× 빈도

METRIC 2

품질 향상

코드 리뷰 발견 이슈 수, PR
합병 후 버그 비율

METRIC 3

비용 추적

월별 토큰 사용량, 모델별 비
용 분포, ROI 계산

METRIC 4

안정성

실패율, 평균 응답 시간, 에
러율, 가용성

MEASUREMENT IS KEY

도입 효과를 숫자로 측정하지 않으면 가치 입증에 어렵습니다. 처음부터 메트릭을 정의하고 추적하세요. 데이터 기반 결정이 핵심입니다.

Time

시간

Quality

품질

Cost

비용

Stability

안정성

사례와 영감

다른 팀의 활용 예시

팀	활용	효과
백엔드	PR 자동 리뷰 + 보안 감사	리뷰 시간 60% 절감
프론트엔드	i18n 자동 번역 + 디자인 일관성	번역 비용 80% 절감
데이터	쿼리 최적화 + 보고서 생성	쿼리 작성 시간 50%
데브옵스	인프라 IaC 검토 + 알람 분석	장애 대응 30% 빠름
PM	이슈 트리아지 + 매주 요약	관리 시간 70% 절감
QA	테스트 케이스 생성 + 버그 분류	커버리지 25% 증가

6 teams

사례

Diverse

다양

Measurable

측정

Start small

시작

학습 자원

계속 깊이 있게 학습하기

Official Documentation

<https://code.claude.com/docs/cli-reference>
<https://code.claude.com/docs/headless>
<https://code.claude.com/docs/automation>
<https://code.claude.com/docs/ci-integration>

GitHub Repositories

github.com/anthropics/claude-code
github.com/anthropics/claude-code-examples
github.com/anthropics/claude-code-action

jq 학습

<https://jqplay.org/> # 인터랙티브
<https://stedolan.github.io/jq/manual/>
<https://github.com/stedolan/jq/wiki/Cookbook>

Shell Scripting

<https://google.github.io/styleguide/shellguide.html>
<https://github.com/dylanaraps/pure-bash-bible>
ShellCheck (정적 분석): [shellcheck.net](https://github.com/koalaman/shellcheck)

GitHub Actions

<https://docs.github.com/actions>
<https://github.com/marketplace?type=actions>

GitLab CI

<https://docs.gitlab.com/ee/ci/>
<https://gitlab.com/components>

CI/CD 베스트 프랙티스

<https://12factor.net/>
<https://martinfowler.com/articles/continuousIntegration.html>

REFERENCES

Docs

공식

jq

파싱

Shell

스크립팅

CI

플랫폼

마지막 팁

오래 활용할 수 있는 습관

TIP 1

작게 시작

거대한 자동화 시도 금지.
가장 자주 반복하는 작업 1개
부터.

TIP 2

측정과 개선

메트릭 없이 운영하지 마세
요.
데이터로 가치를 검증.

TIP 3

팀 공유

본인 스크립트를 팀과 공유.
모두가 혜택을 봐야 정착.

TIP 4

안전 우선

권한, 비용, 보안 통제가 혼
들리면 자동화는 짐.
항상 안전망.

GOLDEN RULE

CLI 자동화의 최종 목표는 사람의 가치 있는 일을 더 많이 할 수 있게 하는 것입니다. AI가 반복 작업을, 사람이 창의적인 일을 합니다.

Small

시작

Measure

측정

Share

공유

Safe

안전

Thank You!

Chapter 5 / CLI Reference

Choi, WooHyung

whchoi@amazon.com

linkedin.com/in/woohyungchoi